

LEARN
PRACTICE
SOLVE
ACE INTERVIEWS



100

Real Scenarios
Real Problems
Real Solutions

REAL-WORLD

DEVOPS

INTERVIEW SCENARIOS

PRACTICAL QUESTIONS. REAL INCIDENTS. EXPERT ANSWERS.



LINUX



NETWORKING



GIT



CI/CD



DOCKER



KUBERNETES



CLOUD



TERRAFORM



SECURITY



MONITORING

- ✓ REAL PRODUCTION SCENARIOS
- ✓ STEP-BY-STEP THINKING
- ✓ DIAGNOSTIC COMMANDS
- ✓ PRACTICAL SOLUTIONS

- ✓ COVERS LINUX TO CLOUD
- ✓ BUILDS TROUBLESHOOTING SKILLS
- ✓ PERFECT FOR INTERVIEWS
- ✓ DESIGNED FOR JUNIOR TO SENIOR

Think Like an Engineer
Solve Like a DevOps Pro

PREPARE
PRACTICE
GET HIRED

VERIQTA

DEVOPS | CLOUD | SRE | PLATFORM ENGINEERING

How to Answer Scenario-Based DevOps Interviews

Think Clearly. Troubleshoot Systematically. Communicate Like an Engineer.

Real
Scenarios
Real Skills
Real Jobs

Scenario 1

An application suddenly becomes unavailable.
Where do you start?



What the Interviewer Wants to Hear

- A structured approach
- Focus on impact and scope
- Check system components in the right order
- Use real commands/tools
- Communicate clearly

Key Steps to Take

- 1 Confirm the impact (who is affected?)
- 2 Check the application status
- 3 Check infrastructure (servers, containers, pods)
- 4 Check logs and recent changes
- 5 Identify the root cause and suggest a fix

Useful Commands

```
>_
ping <host>
curl -I <url>
kubectl get pods
kubectl logs <pod>
systemctl status <service>
docker ps
tail -n 100 /var/log/app.log
```

Common Causes



- Server or pod down
- Application crash
- High resource usage
- Database unavailable
- Network or DNS issue
- Recent bad deployment
- Configuration change

Scenario 2

A deployment completed, but users report errors.
What do you investigate first?



What the Interviewer Wants to Hear

- You verify the deployment
- You check application health
- You compare old vs new version
- You check logs and metrics
- You roll back if needed

Investigation Steps

- 1 Confirm the deployed version
- 2 Check application logs
- 3 Compare configuration changes
- 4 Check dependencies (database, APIs, services)
- 5 Validate metrics and error rates
- 6 Roll back if the new release is the cause

Useful Commands

```
>_
kubectl rollout status <deploy>
kubectl get pods -o wide
kubectl logs <pod> --tail=100
kubectl describe pod <pod>
docker logs <container>
git log --oneline | head
kubectl rollout undo <deploy>
```

Common Issues



- Bad configuration
- Missing environment variables
- Database migration failure
- Incompatible dependency
- Health check failures
- Frontend/backend mismatch
- Traffic not reaching new pods

Scenario 3

You inherit a production incident with almost no documentation.
How do you approach it?



What the Interviewer Wants to Hear

- You stay calm and methodical
- You gather information quickly
- You ask the right questions
- You use available tools
- You communicate clearly
- You create documentation after resolving the issue

Approach

- 1 Understand the current impact
- 2 Identify key components (application, infrastructure, data)
- 3 Gather evidence (logs, metrics, status)
- 4 Form a hypothesis and test it
- 5 Fix the issue or engage the right team
- 6 Document your findings and actions
- 7 Recommend long-term improvements

Key Questions to Ask



- What is not working?
- When did it start?
- What changed recently?
- Who is affected?
- Are there alerts or logs?
- What are the dependencies?
- Is there a rollback option?



Pro Tip

In real DevOps work, you will not always have documentation. Your ability to stay calm, ask the right questions, use the available data and communicate clearly is what makes you valuable.

REAL ENGINEERS SOLVE PROBLEMS



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

LEARN
BUILD
SHARE
GROW

Linux Files, Permissions, and Ownership

Understand Linux permissions. Fix access issues. Think like a DevOps engineer.

Scenario 4 Application returns **Permission denied**.

Problem / Symptoms



- Error: Permission denied
- Application cannot read or execute a file
- Common with config files, scripts or directories

What to Check

- 1 Check file permissions
- 2 Check file ownership
- 3 Confirm the path is correct
- 4 Check if the required user has access
- 5 Check directory permissions (for execute/traverse)

Useful Commands



```
ls -l <file>
stat <file>
namei -l <path>
id
getfacl <file>
sudo -u <user> ls -l <file>
```

Fix / Resolution



- Give correct permissions (e.g. chmod 644 file)
- Set correct ownership (e.g. chown user:group file)
- Ensure parent directories have execute permission
- Follow least privilege principle

Scenario 5 Deployment works with **sudo** but fails as the service user.

Problem / Symptoms



- Command works with sudo
- Fails when run as the application or service user
- Usually a permissions, environment or path issue

What to Check

- 1 Compare environment (sudo vs service user)
- 2 Check file and directory ownership
- 3 Check required permissions
- 4 Check if the service user has the correct PATH
- 5 Review service configuration

Useful Commands



```
sudo -u <user> whoami
sudo -u <user> env
ls -l <file>
id <user>
ps aux | grep <service>
systemctl cat <service>
```

Fix / Resolution



- Set correct permissions and ownership
- Use full paths in scripts
- Match environment variables for the service user
- Update service configuration
- Avoid using sudo in application code

Scenario 6 SSH rejects a private key because its **permissions are too open**.

Problem / Symptoms



- Error: Permissions too open
- SSH refuses to use the key
- Common error: "Bad permissions. Try removing permissions for group and others."

What to Check

- 1 Check current permissions
- 2 Verify file ownership
- 3 Ensure correct key format
- 4 Confirm you are using the right key
- 5 Check the ssh client logs if needed

Useful Commands



```
ls -l ~/.ssh/id_rsa
chmod 600 ~/.ssh/id_rsa
chown $USER:$USER ~/.ssh/id_rsa
ssh -v -i ~/.ssh/id_rsa user@host
```

Fix / Resolution



- Set correct permissions: chmod 600 <key>
- Ensure correct ownership
- Use the correct key file
- Remove group/other access
- Verify with: ssh -v

Scenario 7 Application **cannot write** to its log directory.

Problem / Symptoms



- Application fails to start or logs show permission errors
- Log file is not created or not updated
- Common in containers and system services

What to Check

- 1 Check directory permissions
- 2 Check directory ownership
- 3 Confirm the service user
- 4 Check if the filesystem is read-only
- 5 Check disk space and inodes

Useful Commands



```
ls -ld /var/log/<app>
touch /var/log/<app>/test.log
id <service-user>
df -h /var/log
mount | grep /var/log
```

Fix / Resolution



- Give write permission to the correct user/group
- Create the directory if missing
- Set correct ownership
- Ensure the filesystem is not read-only
- Verify the application can now write logs



- ✓ Understand Linux permissions (read, write, execute)
- ✓ Always check ownership and the user running the process
- ✓ Use real commands to investigate, not assumptions
- ✓ Follow least privilege and secure permission practices



- ✓ Do not use chmod 777
- ✓ Do not run applications as root
- ✓ Always verify the fix
- ✓ Check logs if something still fails

**BUILD
PRACTICE
GROW**



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

Linux Processes and Services

Understand process behaviour. Manage services. Troubleshoot like a DevOps engineer.

Scenario 8 Application process disappears shortly after starting.

Problem / Symptoms



- Process starts but exits after a few seconds
- Application is not running
- No error message on screen
- Common with misconfiguration, missing dependencies or crashes

What to Check

- Check the process status
- Check application logs
- Run the command manually to see the error
- Check dependencies and environment variables
- Check system resources (memory, disk)

Useful Commands



```
ps -ef | grep <app>
pgrep -f <app>
journalctl -u <service>
./<app> (run manually)
ls -l /path/to/app
ldd <binary>
df -h
free -h
```

Fix / Resolution



- Fix the root cause (e.g. config, dependency)
- Set required environment variables
- Ensure correct permissions
- Update or install missing dependencies
- Use a process manager (systemd, supervisord)
- Verify the process stays running

Scenario 9 systemctl reports a service as failed.

Problem / Symptoms



- systemctl shows the service as failed
- Service is not running
- Application is unavailable
- Error usually in logs or configuration

What to Check

- Check service status
- View recent logs
- Check configuration file
- Look for missing files, permissions or dependencies
- Check if the service user has required access

Useful Commands



```
systemctl status <service>
journalctl -u <service> -n 50
systemctl restart <service>
cat /etc/systemd/system/<service>.service
ls -l /etc/<app>/
ps -ef | grep <app>
```

Fix / Resolution



- Fix configuration errors
- Give correct permissions
- Install missing dependencies
- Update the service file if needed
- Reload and restart the service
`systemctl daemon-reload`
`systemctl restart <service>`
- Verify with systemctl status

Scenario 10 A process consumes nearly 100 percent CPU.

Problem / Symptoms



- CPU usage is very high (near 100%)
- System becomes slow or unresponsive
- Could be a runaway process, infinite loop or high traffic
- Common in applications, scripts or containers

What to Check

- Identify the process
- Check what the process is doing
- Review application logs
- Check if it is expected behaviour (e.g. high traffic)
- Look for code issues, loops or resource leaks

Useful Commands



```
top
htop
ps -eo pid,ppid,cmd,%cpu --sort=-%cpu
top -p <pid>
strace -p <pid>
lsof -p <pid>
kill <pid> (if needed)
```

Fix / Resolution



- Identify and fix the cause (e.g. code loop, heavy query)
- Optimize the application
- Restart the process if safe
- Set resource limits (e.g. in systemd or containers)
- Monitor to ensure it does not happen again



Key Takeaways

- Use the right tools to inspect processes and services
- Logs are your best friend
- Understand how systemd manages services
- Always verify the root cause before applying a fix
- Monitor resource usage to prevent issues



Remember

- A failed service always has a reason
- High CPU or memory often indicates a problem
- Do not blindly restart services
- Check logs, configuration and dependencies
- Fix the cause, not just the symptom

**BUILD
PRACTICE
GROW**



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

Linux Memory, Disk, and Resource Problems

Diagnose resource issues. Keep your systems healthy. Think like a DevOps engineer.

Scenario 11 Server runs out of disk space.

Problem / Symptoms



- Error: No space left on device
- Applications fail to write files
- Logs cannot be written
- Services may crash
- Disk usage shows 100%

What to Check

- Check disk usage
- Find large files and directories
- Check log files
- Check if a specific filesystem is full (e.g. /var, /home)
- Identify what is growing

Useful Commands



```
df -h
du -sh /* 2>/dev/null
du -sh /var/*
find / -type f -size +100M
ls -lh /var/log
journalctl --disk-usage
```

Fix / Resolution



- Remove unnecessary files
- Rotate or clean up logs
- Clear package cache (apt/yum)
- Move or resize storage if needed
- Set up log rotation and monitoring

Scenario 12 Memory usage keeps increasing.

Problem / Symptoms



- Memory usage gradually increases over time
- Server becomes slow
- Application responses degrade
- May eventually lead to OOM (Out of Memory)

What to Check

- Check overall memory usage
- Identify top memory consumers
- Check for memory leaks in the application
- Review logs and application metrics
- Determine if the growth is expected (cache, buffer, etc.)

Useful Commands



```
free -h
top
htop
ps -eo pid,ppid,cmd,%mem --sort=-%mem
smem -r (if installed)
journalctl -u <service> -f
```

Fix / Resolution



- Restart the application (if safe)
- Fix memory leak in code
- Adjust application settings (e.g. heap size)
- Use resource limits (cgroups, Kubernetes)
- Monitor memory usage over time

Scenario 13 A process is killed by the Linux OOM killer.

Problem / Symptoms



- Process suddenly stops
- Application becomes unavailable
- Found in logs: "Killed process <name>" or "Out of memory"
- System may still be running

What to Check

- Check system memory status
- Look at kernel logs (dmesg)
- Identify which process was killed
- Check memory usage patterns
- Verify if swap is enabled

Useful Commands



```
dmesg | grep -i oom
journalctl -k | grep -i oom
free -h
top
ps -eo pid,ppid,cmd,%mem
cat /proc/<pid>/status
```

Fix / Resolution



- Increase available memory (RAM or swap)
- Optimize the application memory usage
- Set memory limits (Kubernetes/cgroups)
- Fix memory leak if present
- Monitor and set up alerts

Scenario 14 Inodes are exhausted even though disk capacity remains.

Problem / Symptoms



- Error: No space left on device
- Cannot create new files even though df shows space
- Applications fail to write logs
- Common on systems with many small files

What to Check

- Check inode usage
- Find directories with many files
- Identify unnecessary files
- Check for application or log file explosion
- Determine why so many files are being created

Useful Commands



```
df -i
find / -xdev -type f | wc -l
du -sh /*
find / -xdev -type f -printf "%s" | wc -l
ls -l | wc -l
```

Fix / Resolution



- Remove unused files
- Clean up old logs and temp files
- Fix application generating excessive files
- Use log rotation
- Consider increasing inode capacity (new filesystem)

Key Takeaways

- Monitor disk, memory and inode usage regularly
- Logs and system tools are your best friends
- Identify the root cause, not just the symptom
- Set up alerts before issues impact users

Remember

- High resource usage often has a reason
- Check logs for the real story
- Monitor, alert and plan for growth
- Prevention is cheaper than downtime

**BUILD
PRACTICE
GROW**



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

LEARN
SHARE
GROW

Linux Logs and Production Investigation

Find the real cause. Use the right logs. Troubleshoot like a DevOps engineer.

Scenario 15 Service fails, but the reason is not obvious.

Problem / Symptoms



- Service is not starting or keeps stopping
- No clear error message
- Application is unavailable
- Logs are unclear or empty
- Happens after a change or reboot

What to Check

- Check service status
- Check recent logs
- Look for configuration errors
- Check dependencies (database, network, etc.)
- Verify environment variables
- Check system logs (for kernel or permission errors)

Useful Commands



```
systemctl status <service>
journalctl -u <service> -n 100
journalctl -xe
tail -n 100 /var/log/<app>.log
grep -i error /var/log/<app>.log
ss -tulnp | grep <port>
```

Fix / Resolution



- Fix the actual error (e.g. config, permission, missing dependency)
- Ensure required services are running
- Start and enable the service
- Validate after fix
- Monitor logs for stability

Scenario 16 Logs suddenly stop appearing.

Problem / Symptoms



- No new logs are being written
- Application seems to be running but logs are empty
- Log file size is not increasing
- May happen after a deployment, rotation or disk issue
- Hard to troubleshoot without logs

What to Check

- Confirm the application is still running
- Check log file path and permissions
- Check disk space and inodes
- Check log rotation configuration
- Verify logging configuration in the application
- Check if logs are being sent to another location (e.g. syslog, journald, Cloud)

Useful Commands



```
ps -ef | grep <app>
ls -l /var/log/<app>.log
df -h
df -i
cat /etc/logrotate.d/<app>
journalctl -u <service>
stat /var/log/<app>.log
ls of | grep <app>.log
```

Fix / Resolution



- Restore correct log path and permissions
- Free up disk space or inodes
- Fix log rotation settings
- Update application logging configuration
- Restart the service if needed
- Verify logs are being written again

Scenario 17 Thousands of errors appear in logs after a release.

Problem / Symptoms



- Sudden spike in error logs
- Happens after a new release deployment
- Application may still be running but with issues
- Users may experience errors
- Logs are flooded and hard to read

What to Check

- Identify the type of errors
- Check what changed in the new release
- Compare with previous version
- Check dependent services and external APIs
- Review application metrics and resource usage
- Look for configuration issues

Useful Commands



```
grep -i error /var/log/<app>.log
journalctl -u <service> -f
tail -f /var/log/<app>.log
grep <error_type> /var/log/<app>.log
kubectl logs <pod> --tail=100
diff <old_config> <new_config>
curl -I <external_api>
top | htop
```

Fix / Resolution



- Identify and fix the root cause (e.g. bad config, code bug, dependency issue)
- Rollback to the previous version if needed
- Apply hotfix or configuration change
- Monitor logs, metrics and user impact after the fix
- Implement better validation and testing before future releases



Key Takeaways

- Logs tell the real story
- Know where to look (application, system, container, cloud)
- Always check recent changes
- Use the right tools and filters
- Monitor and set up proper log rotation and alerts



Remember

- Do not ignore error logs
- Check disk space and inodes
- Validate configuration after every change
- Use structured logging where possible
- Logs are essential for troubleshooting and audits

**BUILD
PRACTICE
GROW**



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

**LEARN
SHARE
GROW**

DNS Failures

Understand DNS issues. Find the root cause. Fix it. Think like a DevOps engineer.

Scenario 18 Application reports Could not resolve host.

Problem / Symptoms



- Error: Could not resolve host
- Application cannot reach external or internal domain
- Name or service not known
- Common in containers, servers or CI/CD pipelines

What to Check

- 1 Check DNS configuration
- 2 Verify network connectivity
- 3 Test name resolution manually
- 4 Check /etc/resolv.conf
- 5 Verify if it is an internal or external domain
- 6 Check if DNS server is reachable

Useful Commands



```
dig <domain>
nslookup <domain>
host <domain>
cat /etc/resolv.conf
systemd-resolve --status
ping <dns-server>
curl -v https://<domain>
```

Fix / Resolution



- Set correct DNS server
- Update /etc/resolv.conf (if needed)
- Fix network or firewall rules
- Use the correct domain name (FQDN)
- Check cloud VPC DNS settings
- Restart the application after fixing

Scenario 19 Domain works from your laptop but not from the server.

Problem / Symptoms



- Domain resolves on your laptop but not on the server
- Application works locally but fails on the server
- Different DNS configuration
- Server may be using internal DNS or no DNS
- Common in cloud or on-prem environments

What to Check

- 1 Compare DNS servers used on both machines
- 2 Check /etc/resolv.conf
- 3 Test resolution from the server
- 4 Check network and firewall rules (port 53)
- 5 Verify if split-horizon DNS is in use
- 6 Check if the server is using a proxy or custom DNS

Useful Commands



```
dig <domain>
nslookup <domain>
cat /etc/resolv.conf
systemd-resolve --status
ping <dns-server>
tracert <domain>
dig @<dns-server> <domain>
```

Fix / Resolution



- Use the correct DNS server
- Update DNS configuration on the server
- Allow DNS traffic (UDP/TCP port 53)
- Configure search domains (if needed)
- Fix network routing issues
- Use the same DNS as your laptop (if appropriate)

Scenario 20 DNS still returns an old IP after migration.

Problem / Symptoms



- DNS resolves to the old IP
- Users still reach the old server
- Changes were made but not reflected
- Caused by DNS caching
- Common after migrations or infrastructure changes

What to Check

- 1 Check the current DNS record
- 2 Test from multiple locations
- 3 Check TTL value
- 4 Clear local DNS cache
- 5 Verify if there are multiple DNS records (A, CNAME)
- 6 Check cloud DNS provider settings

Useful Commands



```
dig <domain>
dig <domain> +nocache
nslookup <domain>
dig <domain> ANY
ipconfig /flushdns (Windows)
systemd-resolve --flush-caches
curl -I <domain>
```

Fix / Resolution



- Wait for TTL to expire
- Manually clear DNS caches
- Update the DNS record correctly
- Remove old records if needed
- Check for multiple records (A, CNAME, etc.)
- Verify the change from multiple networks



Key Takeaways

- ✓ DNS issues are common in real environments
- ✓ Always test from the server, not just your laptop
- ✓ Check DNS configuration, network and firewall rules
- ✓ Be aware of DNS caching and TTL
- ✓ Use the right tools to verify and troubleshoot



Remember

- ✓ DNS uses UDP/TCP port 53
- ✓ Changes may take time due to TTL
- ✓ Different environments can use different DNS
- ✓ Always verify the actual IP returned
- ✓ Check both A and CNAME records

**BUILD
PRACTICE
GROW**



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

**LEARN
SHARE
GROW**

Connectivity, Ports, and Routing

Diagnose network connectivity issues. Check ports, firewalls and routing. Think like a DevOps engineer.

Scenario 21 ping works, but the application cannot connect.

Problem / Symptoms



- ping to the server is successful
- Application cannot connect
- Could be a port, firewall or service issue
- Common with databases, web servers and internal services

What to Check

- 1 Check if the application port is open
- 2 Verify the service is running
- 3 Check firewall rules (local and network)
- 4 Test connectivity using the actual application port
- 5 Check for SSL/TLS issues (if using HTTPS)

Useful Commands



```
ping <server>
nc -zv <server> <port>
telnet <server> <port>
ss -tlnp | grep <port>
sudo firewall-cmd --list-all
sudo iptables -L -n
curl -v http://<server>:<port>
```

Fix / Resolution



- Open the required port in firewall
- Ensure the service is running
- Allow the port in security groups (cloud)
- Fix any SSL/TLS issues
- Verify end-to-end connectivity with the application

Scenario 22 Connection refused appears when accessing a service.

Problem / Symptoms



- Error: Connection refused
- Target server is reachable but actively refusing the connection
- Service may not be running
- Wrong port or IP
- Firewall may be rejecting the connection

What to Check

- 1 Check if the service is running
- 2 Verify the correct port
- 3 Check if the service is listening on the expected interface (0.0.0.0 or specific IP)
- 4 Check firewall rules
- 5 Review service logs

Useful Commands



```
ss -tlnp | grep <port>
systemctl status <service>
netstat -tlnp | grep <port>
nc -zv <server> <port>
telnet <server> <port>
sudo firewall-cmd --list-all
journalctl -u <service> -n 50
```

Fix / Resolution



- Start the service
- Use the correct port and IP
- Update firewall rules
- Configure the service to listen on the correct interface
- Check and fix service configuration
- Verify with a successful connection

Scenario 23 Connection timed out appears instead.

Problem / Symptoms



- Error: Connection timed out
- No response from the target
- Could be network, firewall, routing or security group issue
- Service may be down or dropped by firewall
- Common in cloud and cross-network setups

What to Check

- 1 Check basic network connectivity
- 2 Verify the route to the destination
- 3 Check firewall or security group rules
- 4 Confirm the service is reachable from the network
- 5 Look for network ACL or intermediate device blocking
- 6 Test from another network (if possible)

Useful Commands



```
ping <server>
traceroute <server>
mtr <server>
nc -zv <server> <port>
telnet <server> <port>
sudo ip route
curl -v <server>:<port>
```

Fix / Resolution



- Fix routing issues
- Allow the traffic in firewalls and security groups
- Check network ACLs
- Ensure the service is running and reachable
- Verify there are no intermediate devices blocking the traffic
- Test again from the source network

Scenario 24 Two servers in different networks cannot communicate.

Problem / Symptoms



- Servers cannot reach each other
- ping fails or times out
- May be a routing, firewall, VPN or network ACL issue
- Common in hybrid, VPC or multi-region environments

What to Check

- 1 Check IP addressing and subnets
- 2 Verify routing between networks
- 3 Check firewall rules on both sides
- 4 Test with ping, traceroute and port
- 5 Confirm VPN, VPC peering or transit gateway configuration
- 6 Check network ACLs and security groups

Useful Commands



```
ping <remote-ip>
traceroute <remote-ip>
telnet <remote-ip> <port>
nc -zv <remote-ip> <port>
ip route
ss -tlnp | grep <port>
curl -v http://<remote-ip>:<port>
```

Fix / Resolution



- Update routing tables
- Allow traffic in firewalls and security groups
- Fix VPN or VPC peering setup
- Update network ACLs
- Verify correct subnet and IP
- Test connectivity again from both servers

Key Takeaways

- ping only checks basic connectivity (ICMP)
- Applications use specific ports and protocols
- Connection refused means the server is reachable but rejecting
- Connection timed out means no response (network/firewall)
- Always check routing, firewalls and security groups

Remember

- ping success does not mean the service is working
- Always test the actual application port
- Check both server and network firewalls
- Verify routing and security groups in cloud environments
- Use the right tools for deeper network troubleshooting

BUILD PRACTICE GROW



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

LEARN
SHARE
GROW

HTTP, TLS, Load Balancers, and Firewalls

Troubleshoot web, TLS, load balancer and firewall issues. Think like a DevOps engineer.

Scenario 25 Website returns HTTP 502.

Problem / Symptoms



- Website shows HTTP 502 Bad Gateway
- Users cannot access the application
- Load balancer is up but returns 502
- Backend service may be unhealthy or unreachable
- Happens after a deployment or configuration change

What to Check

- Check backend service health
- Verify application logs
- Check load balancer target status
- Validate network connectivity between LB and backend
- Check recent deployments or config changes
- Look for resource issues on backend (CPU, memory)

Useful Commands



```
curl -I https://<domain>
curl -v https://<domain>
curl http://<backend-ip>:<port>
kubectl get pods
kubectl describe pod <pod>
aws elbv2 describe-target-health
--target-group-arn <arn>
tail -n 100 /var/log/<app>.log
ss -tulnp | grep <port>
```

Fix / Resolution



- Start or restart the backend service
- Fix application errors
- Ensure the load balancer health checks are passing
- Update target group configuration if needed
- Check and fix network connectivity or firewall rules
- Roll back recent changes if necessary

Scenario 26 HTTPS suddenly reports a certificate error.

Problem / Symptoms



- Browser shows certificate error (e.g. expired, untrusted)
- Users cannot access the website over HTTPS
- May have worked before and suddenly started failing
- Could be caused by an expired, misconfigured or incorrect certificate

What to Check

- Check certificate expiration date
- Verify the certificate matches the correct domain
- Check the certificate chain and issuer
- Look for recent certificate changes or renewals
- Validate SSL/TLS configuration on the server or load balancer

Useful Commands



```
openssl s_client -connect
<domain>:443 -servername
<domain>
openssl x509 -in cert.pem -noout
-text
curl -lv https://<domain>
echo | openssl s_client -connect
<domain>:443 -showcerts
openssl verify -CAfile ca-bundle.crt
cert.pem
```

Fix / Resolution



- Renew the expired certificate
- Install the correct certificate for the domain
- Fix the certificate chain (order of certificates)
- Update load balancer or web server SSL configuration
- Ensure automatic renewal is working (e.g. Let's Encrypt)
- Clear browser cache if needed

Scenario 27 Application works directly but fails through the load balancer.

Problem / Symptoms



- Application works when accessed directly from the server
- Fails or shows errors when accessed through the load balancer
- May return 4xx, 5xx or timeout
- Could be related to health checks, headers, SSL/TLS, network or firewall rules

What to Check

- Compare direct access vs load balancer access
- Check load balancer target health status
- Verify health check configuration
- Check for header or protocol issues (e.g. HTTP vs HTTPS)
- Review firewall and security group rules
- Check application logs for differences

Useful Commands



```
curl -I http://<lb-dns>
curl -I http://<backend-ip>
curl -v https://<lb-dns>
aws elbv2 describe-target-health
--target-group-arn <arn>
kubectl get svc
kubectl describe svc <service>
tcpdump -i eth0 host <lb-ip>
tail -f /var/log/<app>.log
```

Fix / Resolution



- Fix health check configuration (path, port, protocol)
- Ensure the correct protocol (HTTP/HTTPS) is used
- Allow required ports in firewall and security groups
- Check and set the correct headers (e.g. X-Forwarded-For)
- Update load balancer rules and listener configuration
- Verify SSL/TLS settings if using HTTPS



Key Takeaways

- Always test from multiple locations (direct, LB and external)
- Check logs, metrics and health checks
- Validate certificates, ports, protocols and firewall rules
- Compare working vs failing paths
- Document and monitor changes



Remember

- HTTP 502 usually means a backend issue
- Certificate errors are often due to expiry or misconfiguration
- Load balancers can introduce their own failures
- Always check network, firewall and security groups
- Use the right tools and logs to find the real cause



LEARN
SHARE
GROW



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

Git Collaboration Failures

Solve common Git issues. Keep your codebase clean. Think like a DevOps engineer.

Scenario 28 Developer accidentally commits directly to main.

Problem / Symptoms



- Developer pushes directly to main
- No pull request (PR) was used
- Changes may be unreviewed
- Can introduce unstable code into production
- Violates branch protection policies

What to Check

- Check commit history
- Identify what was changed
- Verify if branch protection rules exist
- Check if the change impacts production
- Confirm if the commit needs to be reverted

Useful Commands



```
git log --oneline
git show <commit>
git diff <commit>
git branch -a
git status
git revert <commit>
```

Fix / Resolution



- Revert the commit if needed (git revert)
- Enforce branch protection rules on main
- Require pull requests and code reviews
- Educate the team on the workflow
- Set up pre-receive hooks or repository rules

Scenario 29 Two engineers create a merge conflict.

Problem / Symptoms



- Merge conflict occurs during pull request or merge
- Git cannot automatically merge the changes
- Both engineers modified the same part of a file
- Blocks the deployment
- Can cause delays in the release

What to Check

- Identify the conflicting files
- Review both changes
- Understand the intended changes from both sides
- Check for partial or incorrect merges
- Communicate with the other engineer if needed

Useful Commands



```
git status
git diff
git merge <branch>
git mergetool
git add <file>
git commit
git push
```

Fix / Resolution



- Open the conflicting file
- Resolve the conflict manually
- Test the changes locally
- Commit and push the fix
- Use clear communication to agree on the correct change
- Consider using smaller and more focused commits in the future

Scenario 30 A secret is accidentally committed to Git.

Problem / Symptoms



- API key, password or sensitive file is in the repo
- Secret is visible in commit history
- May be exposed to the public
- Even if deleted later, it can still be recovered from history
- Security and compliance risk

What to Check

- Identify what secret was committed
- Check commit history
- Verify if the repository is public
- Determine if the secret has already been exposed
- Check for other possible secrets in the repo

Useful Commands



```
git log --oneline
git show <commit>
git grep -i "secret"
git filter-repo --path <file> --invert-paths
git push --force
```

Fix / Resolution



- Revoke and rotate the secret immediately
- Remove the secret from the repository history
- Use git filter-repo or BFG
- Update .gitignore to prevent future commits
- Scan the repository for other secrets (e.g. using gitleaks)
- Educate the team on handling sensitive information

Scenario 31 A required production change must be safely reverted.

Problem / Symptoms



- A recent change caused issues in production
- Need to roll back to a previous working version
- Must do it safely without breaking other changes
- Minimize downtime
- Keep a clear audit trail

What to Check

- Identify the commit that introduced the issue
- Review what changes were made
- Check if there are dependent changes
- Confirm the current production version
- Plan the rollback approach

Useful Commands



```
git log --oneline
git show <commit>
git revert <commit>
git push
git tag rollback-<date>
git checkout <tag>
```

Fix / Resolution



- Revert the problematic commit
- Test the revert in a staging environment if possible
- Deploy the rollback during a low-traffic window
- Monitor the application after rollback
- Document the incident and root cause
- Consider adding automated tests to prevent recurrence

Key Takeaways

- Use pull requests and code reviews
- Keep sensitive information out of Git
- Communicate clearly with your team
- Use proper branching strategies
- Always have a rollback plan

Remember

- Git history is permanent if pushed
- Secrets can be recovered even after deletion
- Small, focused commits reduce conflicts
- Test before deploying or reverting
- Good Git practices keep your team and systems safe

**BUILD
PRACTICE
GROW**

Git Recovery and Release Problems

Fix Git mistakes, recover changes, and keep releases stable. Think like a DevOps engineer.

Scenario 32 A bad commit has already been pushed.

Problem / Symptoms



- A commit with bugs, sensitive data or unwanted changes has been pushed
- The code is already on the remote repository
- It may have been pulled by others
- Could break the build or production
- Need to remove or revert safely

What to Check

- Review the commit details
- Check if others have already pulled the commit
- Determine if you should revert or remove the commit (history rewrite)
- Check branch protection rules
- Identify any exposed secrets (if applicable)
- Plan the safest recovery approach

Useful Commands



```
# Revert the bad commit
git revert <commit-hash>

# Reset and force push
(use with caution)
git reset --hard <commit>
git push --force

# Remove sensitive file
from history
git filter-repo --path <file>
--invert-paths
```

Fix / Resolution



- Use git revert for a safe rollback (recommended)
- Use force push only if you are sure and coordinate with your team
- Rotate any exposed credentials immediately
- Update branch protection rules
- Communicate with the team about the fix
- Verify the repository is clean

Scenario 33 CI builds the wrong version of the application.

Problem / Symptoms



- CI builds an older or unexpected version
- The build does not include the latest changes
- Wrong branch, tag or commit is being used
- Artifact version is incorrect
- Deployment may release the wrong code
- Causes confusion and potential production issues

What to Check

- Check which branch, tag or commit the CI is using
- Verify the CI configuration (e.g. Jenkins, GitHub Actions, GitLab CI)
- Check for caching issues
- Review build logs
- Confirm versioning strategy (e.g. semantic versioning)
- Check if the code was actually pushed to the expected branch

Useful Commands



```
git branch -a
git tag -l
git log --oneline -n 10
git show <commit>

# Jenkins example
(in build log)
echo $GIT_BRANCH
echo $GIT_COMMIT
echo $BUILD_VERSION
```

Fix / Resolution



- Update CI configuration to use the correct branch or tag
- Clear build caches
- Ensure versioning is automated and consistent
- Verify the latest code is pushed before the build
- Add checks in the pipeline to validate the version
- Re-run the build and confirm the correct version

Scenario 34 A release branch has diverged significantly from main.

Problem / Symptoms



- Release branch is far behind or ahead of main
- Many conflicting changes
- Difficult or risky to merge
- Hotfixes may be missing in the release branch
- Could cause deployment issues
- Unclear which changes should be included

What to Check

- Check the commit history of both branches
- Identify the differences between branches
- Review missing hotfixes or important changes
- Look for conflicting files
- Confirm which changes should be in the release
- Check if a rebase or merge is the best approach

Useful Commands



```
git branch -a
git log --oneline main
git log --oneline release
git diff main...release
git merge main
# or
git rebase main
git status
```

Fix / Resolution



- Update the release branch with the latest changes (from main)
- Resolve any merge conflicts
- Include missing hotfixes
- Use a clear branching strategy
- Consider rebasing if history is messy
- Test the build after merging
- Tag the release version (e.g. v1.2.0)
- Document the changes



Key Takeaways

- Always review before pushing
- Use revert instead of force push when possible
- Automate versioning in CI/CD
- Keep release branches in sync with main
- Communicate with your team



Remember

- A pushed commit can impact others
- CI must build the correct branch, tag or commit
- Diverged branches increase merge risk
- Use proper Git strategies (revert, rebase, merge)
- Test and verify before releasing



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

**LEARN
SHARE
GROW**

CI Pipeline Failures

Troubleshoot CI issues. Keep your pipeline reliable. Think like a DevOps engineer.

Scenario 35 Pipeline suddenly fails although the code appears unchanged.

Problem / Symptoms

- Pipeline failed without any code changes
- Worked in previous runs
- External dependencies may have changed
- Base image or runner image may have been updated
- Flaky tests or environment issues
- Hard to identify the cause

What to Check

- Compare the current run with the last successful run
- Check for changes in build environment, base images or dependencies
- Review pipeline and job logs
- Check for external service outages (package registries, APIs, etc.)
- Verify cached dependencies
- Look for flaky tests or timing issues

Useful Commands

```
>_
# Check recent pipeline runs
# (GitHub Actions)
gh run list
gh run view <run-id> --log
# Check runner environment
uname -a
docker version
# Check dependency versions
cat package-lock.json
cat requirements.txt
```

Fix / Resolution

- Rerun the pipeline to confirm it is not a transient issue
- Pin dependency versions
- Use fixed base image tags (instead of latest)
- Clear caches and rebuild
- Check and update external service credentials
- Add retry logic for flaky tests
- Set up alerts for dependency or service outages

Scenario 36 Tests pass locally but fail in CI.

Problem / Symptoms

- All tests pass on the developer machine
- Tests fail in the CI environment
- Environment differences (OS, versions, dependencies)
- Missing environment variables or secrets
- Tests may be order-dependent or flaky
- Works on my machine issue

What to Check

- Compare local and CI environments
- Check dependency versions
- Verify environment variables and secrets
- Review test logs and error messages
- Check for database, network or external service dependencies
- Look for tests that rely on local files or configuration

Useful Commands

```
>_
# Check environment
python --version
node --version
java -version
# Compare installed packages
pip freeze
npm list
# Run tests locally with same environment as CI (Docker)
docker run -it <image>
```

Fix / Resolution

- Use the same runtime versions as CI
- Pin and lock dependencies
- Set required environment variables and secrets in CI
- Use Docker to replicate the CI environment locally
- Fix unstable or order-dependent tests
- Update test data and mocks
- Ensure required services (e.g. database) are available in CI

Scenario 37 Build succeeds on one runner but fails on another.

Problem / Symptoms

- Build works on one runner but fails on another
- Different runner OS, architecture or configuration
- Missing tools or dependencies
- Inconsistent environment setup
- May be caused by self-hosted runners or different runner images

What to Check

- Compare runner environments
- Check OS, architecture and installed tools
- Verify pipeline configuration
- Check for missing dependencies
- Review build logs for specific errors
- Ensure the build is not relying on local state

Useful Commands

```
>_
# Check system info on runner
uname -a
cat /etc/os-release
docker --version
git --version
# Check installed tools
which <tool>
<tool> --version
```

Fix / Resolution

- Use consistent runner images
- Install required tools and dependencies
- Standardize the build environment (use Docker)
- Update pipeline configuration
- Use infrastructure as code for self-hosted runners
- Document runner requirements
- Test the build on a clean environment

Scenario 38 Pipeline cannot download a dependency.

Problem / Symptoms

- Pipeline fails while downloading a dependency
- Timeout, network or DNS issues
- Repository or registry unavailable
- Authentication or permission errors
- Firewall or proxy blocking access
- Rate limiting from package registry

What to Check

- Check network connectivity
- Verify the package repository URL
- Check DNS resolution
- Validate authentication credentials (tokens, keys)
- Check firewall or proxy settings
- Look for rate limiting or quota issues

Useful Commands

```
>_
# Test connectivity
dig <registry-domain>
curl -I https://<registry>
# Test download
curl -v https://<package-url>
# Check proxy settings
env | grep -i proxy
```

Fix / Resolution

- Ensure network and DNS are working
- Use the correct repository URL
- Configure and verify credentials
- Set or update proxy settings
- Use a mirror or internal registry
- Increase timeout and retry settings
- Check and resolve rate limits (e.g. use authentication or caching)

Key Takeaways

- CI failures are often environment or dependency related
- Keep environments consistent and reproducible
- Use pinned versions and reliable sources
- Check logs and compare successful vs failed runs
- Monitor and alert on pipeline failures

Remember

- Don't rely on "it works on my machine"
- Use version control for your pipeline configuration
- Keep secrets secure and up to date
- Document and standardize your build environment
- Invest in reliable and repeatable pipelines

BUILD PRACTICE GROW

Deployment Pipeline Failures

Troubleshoot deployment issues. Ensure reliable and safe releases. Think like a DevOps engineer.

Scenario 39

Build succeeds but deployment fails.

Problem / Symptoms



- Build completes successfully but deployment step fails
- Error during deployment (e.g. permission, config, resource issue)
- Application is not updated in the target environment
- Rollback may be triggered automatically
- Common in Kubernetes, VMs, serverless or PaaS

What to Check

- Check deployment logs in the CI/CD tool
- Verify environment configuration (variables, secrets, permissions)
- Ensure the target environment is reachable
- Check resource limits (CPU, memory, quotas)
- Validate deployment manifests (e.g. Kubernetes YAML, Docker image tag)
- Look for recent infrastructure or policy changes

Useful Commands



```
# Check deployment logs
kubectl rollout status deployment/<app>
kubectl describe pod
kubectl get events

# Check service status
kubectl get pods
kubectl logs <pod>

# Check image tag
docker images | grep <app>
```

Fix / Resolution



- Check and fix the root cause in the deployment logs
- Verify all required secrets and environment variables are set
- Ensure the target environment has enough resources
- Fix permission or access issues
- Update and reapply the deployment configuration
- Redeploy after fixing the issue
- Use a rollback if a partial deployment occurred

Scenario 40

Deployment reports success, but the application is unhealthy.

Problem / Symptoms



- CI/CD pipeline reports deployment as successful
- Application is not working as expected
- Health checks fail
- Users experience errors or downtime
- New version may be running but not functioning correctly
- Can be caused by bad config, migration issues or runtime errors

What to Check

- Verify application health (health endpoints, logs)
- Check if all instances are running correctly
- Validate configuration and environment variables
- Review recent application logs
- Check database migrations or external dependencies
- Compare the deployed version with the expected version

Useful Commands



```
# Check pod status
kubectl get pods
# Check health endpoint
curl -I http://<app>/health
# View application logs
kubectl logs <pod> --tail 100
# Check service and endpoints
kubectl get svc
kubectl get endpoints
# Check application metrics
kubectl top pods
```

Fix / Resolution



- Identify the root cause from logs and health checks
- Fix configuration or environment variable issues
- Verify all required dependencies are available
- Check database migrations
- Redeploy with the correct fix
- Roll back if the current version is not working
- Add or improve health checks to catch issues earlier

Scenario 41

Production receives the wrong artifact version.

Problem / Symptoms



- Unexpected or old version is deployed to production
- Version does not match the commit or release tag
- Users see old features or missing changes
- Can happen due to caching, wrong tag, or pipeline issue
- May cause confusion and rollback

What to Check

- Verify the deployed version in the application
- Check the artifact version in the CI/CD pipeline
- Review image tags or package versions
- Confirm the correct branch, tag or commit was used
- Check for caching issues in the registry or CDN
- Review deployment logs and pipeline configuration

Useful Commands



```
# Check application version
kubectl exec <pod> -- /app --version
# Check image tag
kubectl get pods -o wide
docker images | grep <app>
# Check pipeline logs
# (use your CI tool UI)
# Check registry tags
docker pull <image>:<tag>
```

Fix / Resolution



- Identify the correct version from source control
- Update the pipeline to use the correct tag or commit
- Clear caches in CI/CD, registry or CDN
- Verify artifact promotion process
- Redeploy with the correct version
- Add version checks and approvals to the pipeline
- Document and communicate the incident

Key Takeaways

- Always verify what is deployed
- Use versioning and tags correctly
- Monitor application health after deployment
- Have a rollback plan ready
- Automate and validate your deployment pipeline

Remember

- A successful pipeline does not always mean a healthy application
- Always confirm the deployed version
- Use health checks, logging and monitoring
- Keep deployments repeatable and auditable

BUILD PRACTICE GROW

LEARN
SHARE
GROW



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

CI/CD Security and Reliability

Secure your pipeline. Prevent failures. Keep your releases reliable. Think like a DevOps engineer.

Scenario 42 Pipeline credential appears in logs.

Problem / Symptoms	What to Check	Useful Commands	Fix / Resolution
 - Sensitive credential (token, password, key) is visible in pipeline logs - Can be accessed by anyone with log access - May be exposed in CI/CD UI, artifacts or notifications - Serious security risk - Requires immediate action	- Identify which credential was exposed - Check the pipeline logs and job history - Determine where the credential came from (env variable, script, config file) - Check if the credential is still active - Look for other places it may have been exposed - Review secret handling practices in the pipeline	 <pre># View pipeline logs # (GitHub Actions) gh run view <run-id> --log # Check recent runs gh run list # Search for secrets in logs grep -i "AKIA\ token\ pass" build.log # Revoke exposed credentials (use your cloud provider console)</pre>	 - Revoke or rotate the exposed credential immediately - Remove the secret from logs (history if possible) - Use secret masking features (e.g. GitHub Actions secrets) - Store secrets in a proper secret manager (e.g. AWS Secrets Manager, HashiCorp Vault) - Review and update pipeline scripts to avoid printing sensitive data - Implement least-privilege access

Scenario 43 CI runner has excessive production permissions.

Problem / Symptoms	What to Check	Useful Commands	Fix / Resolution
 - CI runner can access or modify production resources - Runner has more permissions than required - Risk of accidental or malicious changes - Violates security best practices - Can lead to data loss or service outage	- Review the runner's IAM or access credentials - Check what resources and actions it can access - Validate if production access is really required - Look for over-permissive policies (e.g. wildcards) - Check audit logs for recent runner activity - Compare with the principle of least privilege	 <pre># (AWS) Check attached policies aws iam list-attached-role-policies --role-name <role> aws iam get-policy-version --policy-arn <arn> --version-id v1 # Check recent activity aws cloudtrail lookup-events --lookup-attributes AttributeKey=PrincipalName, AttributeValue=<runner-role></pre>	 - Remove unnecessary permissions - Use separate roles for dev, staging and production - Apply the principle of least privilege - Use environment-specific credentials - Implement approval checks for production actions - Monitor and alert on runner activity - Regularly review and audit permissions

Scenario 44 Deployment is interrupted halfway through.

Problem / Symptoms	What to Check	Useful Commands	Fix / Resolution
 - Deployment stops or fails midway - Some resources are updated, others are not - Application may be in an inconsistent state - Service may be partially available or completely down - Can be caused by network issues, timeouts, credit limits, or errors in scripts	- Check deployment logs to find where it stopped - Verify the current state of the application and resources - Look for error messages or timeouts - Check if rollback was triggered automatically - Validate infrastructure limits (e.g. quotas, disk space) - Identify if manual intervention is required	 <pre># Check deployment logs kubectl rollout status deployment/<app> kubectl describe deployment <app> # Check pod status kubectl get pods # Check recent events kubectl get events --sort-by=.lastTimestamp # Check application health curl -I http://<service-endpoint></pre>	 - Identify and fix the root cause (e.g. network, quota, config) - Complete the deployment or roll back to a stable version - Use automated rollback mechanisms (e.g. Kubernetes) - Increase timeouts or resource limits if needed - Implement more robust deployment strategies (e.g. blue/green or canary) - Verify application health after recovery

Scenario 45 A new release must be rolled back safely.

Problem / Symptoms	What to Check	Useful Commands	Fix / Resolution
 - New release causes issues (e.g. errors, performance, outages) - Need to return to a previous stable version - Rollback must be done safely without causing more downtime - Data or configuration changes may also need to be reverted - Users are impacted	- Identify the last known good version - Check the current state and impact of the release - Review rollback options (automatic or manual) - Verify if database or config changes also need to be reverted - Plan the rollback steps and communication - Validate the application after rollback	 <pre># Kubernetes rollback kubectl rollout undo deployment/<app> kubectl rollout history deployment/<app> # Docker/Compose docker compose down docker compose up -d <previous-tag> # Check application status kubectl get pods kubectl get svc</pre>	 - Rollback to the previous stable version - Revert database or configuration changes if required - Follow a documented rollback procedure - Communicate with stakeholders about the rollback - Monitor the application after rollback to ensure stability - Conduct a post-incident review to prevent recurrence

Key Takeaways

- Keep your pipelines secure and your credentials protected
- Use least-privilege access for CI/CD runners
- Always have a rollback plan
- Monitor your deployments and set up proper alerts
- Learn from failures and continuously improve

Remember

- Secrets in logs are a serious security risk
- Over-permissioned runners can cause production incidents
- Incomplete deployments can leave systems in a bad state
- Rollback quickly and safely when needed
- Security and reliability are everyone's responsibility

**BUILD
PRACTICE
GROW**

Docker Container Failures

Troubleshoot container issues. Keep your services running. Think like a DevOps engineer.

Scenario 46 Container exits immediately after starting.

Problem / Symptoms	What to Check	Useful Commands	Fix / Resolution
 - Container starts and stops immediately - Exit code is non-zero (e.g. 1, 127, 126) - Application does not stay running - May be due to wrong command, missing file, permission issue or application error - Container shows as "Exited" in docker ps	- Check container logs - Check the exit code - Verify the startup command (CMD / ENTRYPOINT) - Confirm required files and dependencies exist - Check file permissions - Test the command manually inside the container	 <pre>docker ps -a docker logs <container> docker inspect <container> docker inspect <container> grep -i exitcode docker run -it <image> /bin/bash docker run --entrypoint /bin/sh <image></pre>	 - Fix the application error in the logs - Correct the CMD or ENTRYPOINT - Ensure required files and dependencies are included in the image - Fix file permissions - Use the correct user (e.g. non-root) - Rebuild the image and test again

Scenario 47 Container continuously restarts.

Problem / Symptoms	What to Check	Useful Commands	Fix / Resolution
 - Container is in a restart loop - Status shows "Restarting" - Application keeps crashing - Can be caused by application errors, health check failures, resource limits or misconfiguration - Logs may show repeated errors	- Check container logs for errors - Verify the restart policy - Check resource usage (CPU, memory) - Validate health check configuration - Look for external dependencies (database, API, etc.) - Check recent changes in the image or configuration	 <pre>docker ps docker logs <container> docker inspect <container> grep -i restart docker stats <container> docker inspect <container> grep -A 10 Health docker events --filter container=<container></pre>	 - Fix the root cause in the application - Adjust or remove the restart policy if needed - Increase resource limits - Fix or disable failing health checks - Ensure external services are reachable - Rebuild and redeploy the container

Scenario 48 Application works locally but fails inside its container.

Problem / Symptoms	What to Check	Useful Commands	Fix / Resolution
 - Application runs fine on your laptop but fails in the container - Different environment (configuration, dependencies, OS, permissions) - May be due to missing packages, environment variables, network issues or file paths - Common with databases, APIs or file system access	- Compare local environment with container environment - Check environment variables - Verify all dependencies are installed in the image - Check file paths and working directory - Test network connectivity from inside the container - Check user permissions and mounted volumes	 <pre>docker exec -it <container> /bin/sh env sort cat /etc/os-release pip list # or npm list ls -la pwd curl -v <service-url> docker inspect <container> grep -A 20 Mounts</pre>	 - Ensure all required dependencies are in the Dockerfile - Set the correct environment variables - Use the correct file paths and working directory - Configure network access (e.g. --network, DNS, proxies) - Fix permissions or run as the correct user - Update the Dockerfile, rebuild the image and test again


Key Takeaways

- Always check container logs first
- Understand the exit code and restart policy
- Keep your Dockerfile clean and reproducible
- Match environments (local vs container)
- Use health checks and monitoring


Remember

- Logs tell the real story
- Small configuration mistakes can break containers
- Test inside the container, not just on your machine
- Use version control for your Dockerfiles
- A working container is a reliable container

**BUILD
PRACTICE
GROW**

**LEARN
SHARE
GROW**

Docker Images and Builds

Build better images. Fix build issues. Keep your Docker workflow reliable. Think like a DevOps engineer.

Scenario 49 Docker image suddenly becomes enormous.

Problem / Symptoms



- Image size is much larger than expected
- Build takes a long time
- Push and pull are slow
- Unnecessary files are included in the image
- Image contains build tools, cache, or temporary files

What to Check

- Check the Dockerfile for unnecessary files and layers
- Look for large dependencies or base image size
- Verify .dockerignore is configured correctly
- Check if build cache is being used efficiently
- Inspect image layers to find what is taking space
- Review if multi-stage build can be used

Useful Commands



```
docker images
docker history <image>
docker image inspect <image>
docker system df
docker build --no-cache .
docker build --progress=plain .
```

Fix / Resolution



- Use a smaller base image (e.g. alpine)
- Add or update .dockerignore
- Use multi-stage builds
- Remove unnecessary files and dependencies
- Clean package manager caches (e.g. apt, yum, npm)
- Combine RUN commands to reduce layers
- Rebuild and verify the image size is reduced

Scenario 50 docker build fails after a dependency change.

Problem / Symptoms



- Build fails during dependency installation
- Error related to a specific package or version
- Worked before but now fails after a dependency change
- Network, repository or permission errors
- Build stops at a specific step

What to Check

- Review the exact error message in the build logs
- Check if the dependency version exists and is compatible
- Verify network access to the package repository
- Check if any required system libraries are missing
- Validate the Dockerfile and build context
- Test the dependency installation outside the build

Useful Commands



```
docker build --no-cache .
docker build --progress=plain .
docker build --pull .
docker run -it --rm <image> sh
pip install <package>
npm install
apt-get update && apt-get install <package>
```

Fix / Resolution



- Use the correct and available dependency version
- Update the package repository or use a different mirror
- Add missing system dependencies
- Fix network, proxy or DNS issues
- Update the Dockerfile with the correct install commands
- Use a stable base image
- Rebuild and confirm the build completes successfully

Scenario 51 Image works on one machine but not another.

Problem / Symptoms



- Container runs fine on one machine but fails on another
- Application behaves unexpectedly
- Missing libraries or dependencies
- Environment differences (OS, architecture, Docker version)
- Works on local machine but not in CI or server

What to Check

- Compare Docker and OS versions on both machines
- Check for platform/architecture differences (e.g. amd64 vs arm64)
- Verify environment variables and configuration files
- Check if the image was built consistently
- Inspect container logs for errors
- Test with the same image on a clean environment

Useful Commands



```
docker --version
docker info
uname -m
docker image inspect <image>
docker run --rm -it <image> sh
docker logs <container>
docker inspect <container>
```

Fix / Resolution



- Use the same base image and Docker version across machines
- Build for the correct platform (e.g. --platform linux/amd64)
- Ensure all required environment variables are set
- Include all dependencies in the image
- Avoid relying on local files or machine-specific configuration
- Rebuild the image and test on the target machine

Scenario 52 Registry rejects an image push.

Problem / Symptoms



- docker push fails with an error
- Permission denied or unauthorized
- Repository not found
- Image tag is invalid
- Image size exceeds limits
- Policy or security scan failure

What to Check

- Verify you are logged in to the correct registry
- Check repository name and tag format
- Confirm you have push permissions
- Check registry limits and quotas
- Look at the full error message
- Verify if the image violates any security or content policy

Useful Commands



```
docker login <registry>
docker tag <image> <registry>/<repo>:<tag>
docker push <registry>/<repo>:<tag>
docker logout
docker image ls
docker system df
```

Fix / Resolution



- Login to the correct registry (e.g. Docker Hub, ECR, ACR, GCR)
- Use the correct repository name and tag format
- Ensure your account has push access
- Reduce image size if it exceeds limits
- Fix any security or policy violations (e.g. remove sensitive content)
- Check and follow registry-specific requirements
- Push again and confirm success

Key Takeaways

- Use .dockerignore to keep images small
- Leverage multi-stage builds
- Keep dependencies updated and secure
- Test images in clean environments
- Follow registry best practices
- Automate image scanning in CI/CD

Remember

- Small, secure, reproducible images
- Same environment for build and run
- Check logs and error messages carefully
- Use the correct registry and permissions
- Automate testing and scanning

**BUILD
PRACTICE
GROW**

Docker Networking, Storage, and Security

Solve common Docker networking, storage and security issues. Build secure and reliable containers.
Think like a DevOps engineer.

Scenario 53 Two containers cannot communicate.

Problem / Symptoms



- Two containers cannot reach each other
- Connection refused or timeout
- Containers may be on different networks
- Wrong container name or IP address
- Application works locally but not between containers
- Firewall or network policy is blocking traffic

What to Check

- Check if both containers are on the same Docker network
- Verify container names and DNS resolution
- Check the service port is running and exposed
- Test connectivity using ping, curl or nc
- Inspect network configuration and driver type
- Check for firewall rules or network policies

Useful Commands



```
docker network ls
docker network inspect <network>
docker ps
docker exec -it <container> sh
ping <container-name>
curl http://<container-name>:<port>
nc -zv <container-name> <port>
docker inspect <container> | grep -i network
```

Fix / Resolution



- Ensure both containers are on the same user-defined network
- Use container names instead of IP addresses
- Expose and use the correct port
- Verify the service is running inside the target container
- Update firewall or network policies if needed
- Test again after making changes

Scenario 54 Container data disappears after recreation.

Problem / Symptoms



- Data created inside the container is lost after recreation
- Application state is gone
- Database files or uploads are missing
- Container works but data does not persist
- Common with databases, logs or uploaded files

What to Check

- Check if a volume is used for persistent storage
- Verify volume is correctly mounted
- Inspect the Dockerfile and container run command
- Check if the container is writing to the correct path
- Verify volume exists and is not removed
- Look for permission issues on the volume

Useful Commands



```
docker volume ls
docker volume inspect <volume>
docker ps -a
docker inspect <container> | grep -i mounts
docker run -v <volume>:<path> <image>
ls -l <host-path>
docker logs <container>
```

Fix / Resolution



- Use Docker volumes for persistent data
- Mount the correct host path or named volume
- Update docker run or docker-compose configuration
- Ensure the application writes to the mounted path
- Check and fix file permissions on the volume
- Recreate the container and verify the data persists

Scenario 55 Container is running as root unnecessarily.

Problem / Symptoms



- Container runs as root user by default
- Security risk and potential privilege escalation
- Fails security scans or compliance checks
- Not following container security best practices
- Should run with non-root user

What to Check

- Check the current user inside the container
- Review the Dockerfile USER instruction
- Verify file and directory permissions
- Check if the application requires root privileges
- Review security scan results
- Identify capabilities that can be dropped

Useful Commands



```
docker exec -it <container> whoami
docker inspect <container> | grep -i user
cat Dockerfile
docker run --rm <image> whoami
id
docker scan <image>
docker run --cap-drop=ALL --security-opt no-new-privileges <image>
```

Fix / Resolution



- Add a non-root user in the Dockerfile
- Use the USER instruction
- Set correct file ownership and permissions
- Drop unnecessary Linux capabilities
- Use read-only filesystems where possible
- Follow container security best practices and re-test



Key Takeaways

- Use user-defined networks for container communication
- Use volumes to persist important data
- Avoid running containers as root
- Implement least privilege and security best practices
- Test and verify after every change



Remember

- Containers are isolated but still need proper configuration
- Data does not persist without a volume
- Running as root is a security risk
- Always verify network, storage and security settings
- Small misconfigurations can cause big issues

**BUILD
PRACTICE
GROW**


@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

**LEARN
SHARE
GROW**

Kubernetes Pod Failures

Troubleshoot pod issues. Keep your clusters healthy and reliable. Think like a DevOps engineer.

Scenario 56 Pod enters CrashLoopBackOff.

Problem / Symptoms



- Pod repeatedly starts and then crashes
- STATUS shows CrashLoopBackOff
- Application exits with error
- Misconfiguration, missing dependencies or code issue
- Logs show the same error repeatedly

What to Check

- Check pod logs for the error
- Verify the container command and arguments
- Check environment variables and configuration files
- Confirm required dependencies and packages are installed
- Look for application errors or exceptions
- Review recent changes (e.g. new image or config)

Useful Commands



```
kubectl get pods
kubectl describe pod <pod>
kubectl logs <pod> --previous
kubectl logs <pod>
kubectl get events --sort-by=.lastTimestamp
kubectl exec -it <pod> -- /bin/sh
```

Fix / Resolution



- Fix the root cause in the application or configuration
- Update environment variables
- Install missing dependencies
- Correct the container command
- Redeploy with a working image
- Monitor logs to confirm the pod stays healthy

Scenario 57 Pod shows ImagePullBackOff.

Problem / Symptoms



- Pod cannot pull the container image
- STATUS shows ImagePullBackOff
- Image not found or access denied
- Incorrect image name or tag
- Missing or invalid image pull secrets
- Network or registry issues

What to Check

- Verify the image name and tag
- Check if the image exists in the registry
- Confirm image pull secrets are configured correctly
- Check registry access and authentication
- Look for network or DNS issues
- Review events for detailed error messages

Useful Commands



```
kubectl describe pod <pod>
kubectl get events
kubectl get secret
kubectl describe secret <name>
kubectl get pods
docker pull <image>
```

Fix / Resolution



- Correct the image name or tag
- Create or update image pull secrets
- Verify registry credentials and permissions
- Check network connectivity to the registry
- Use a reachable and trusted container registry
- Redeploy and confirm the image pulls successfully

Scenario 58 Pod remains Pending.

Problem / Symptoms



- Pod stays in Pending state
- Not enough cluster resources (CPU, memory)
- Node selectors, taints or tolerations issues
- PersistentVolumeClaim (PVC) not bound
- Image pull or scheduling constraints
- Events show scheduling failures

What to Check

- Check pod events for the reason
- Verify cluster resource availability
- Check node selectors, taints and tolerations
- Verify PersistentVolumeClaim status
- Confirm storage class and provisioner
- Review resource requests and limits

Useful Commands



```
kubectl describe pod <pod>
kubectl get nodes
kubectl get pvc
kubectl get storageclass
kubectl get events
kubectl top nodes
```

Fix / Resolution



- Add more cluster resources or scale the cluster
- Fix node selectors, taints or tolerations
- Create or fix the PVC and storage class
- Adjust resource requests/limits
- Remove conflicting constraints
- Redeploy and verify the pod is scheduled and running

Scenario 59 Pod is repeatedly OOMKilled.

Problem / Symptoms



- Pod is terminated with OOMKilled
- Application is using too much memory
- Memory limits are too low
- Memory leak in the application
- Pod restarts repeatedly
- Performance degradation before it is killed

What to Check

- Check pod logs for memory usage
- Review resource limits and requests
- Monitor memory usage over time
- Check for memory leaks in the application
- Analyze application metrics
- Compare with previous working version

Useful Commands



```
kubectl describe pod <pod>
kubectl logs <pod>
kubectl top pod <pod>
kubectl top nodes
kubectl get events
kubectl edit deployment <name>
```

Fix / Resolution



- Increase memory limits and requests
- Optimize the application to use less memory
- Fix memory leaks
- Set appropriate resource limits
- Monitor memory usage regularly
- Redeploy and confirm the pod stays healthy

Key Takeaways

- Always check pod events and logs
- Understand Kubernetes scheduling and resources
- Use proper image names and pull secrets
- Set correct resource requests and limits
- Monitor your pods and fix issues early

Remember

- Read the actual error message
- Check recent changes
- Validate configuration and resources
- Test the fix and monitor
- A healthy cluster runs healthy workloads

BUILD PRACTICE GROW



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

**LEARN
SHARE
GROW**

Kubernetes Services and Networking

Troubleshoot Kubernetes networking and service connectivity issues. Keep your applications reachable. Think like a DevOps engineer.

Scenario 60 Pod is healthy, but the Service cannot reach it.

Problem / Symptoms	What to Check	Useful Commands	Fix / Resolution
 - Pod is running and healthy (Ready state) - Service cannot reach the pod - Connection timeout or "connection refused" - Wrong selector or labels - Service endpoints are empty - Traffic goes to the wrong port - Works when testing inside the pod but not via Service	- Verify pod is running and ready - Check Service selectors match pod labels - Check Endpoints or EndpointSlices - Verify the correct targetPort and port configuration - Test connectivity from another pod using curl or nc - Check for NetworkPolicy blocking traffic	 <pre>kubectl get pods -o wide kubectl get svc kubectl get endpoints <svc> kubectl get endpointslices kubectl describe svc <svc> kubectl get pod --show-labels kubectl run test --rm -it \ --image=busybox -- sh curl http://<service-name>: <port> kubectl get networkpolicy</pre>	 - Ensure Service selector matches pod labels - Verify correct targetPort and port - Fix incorrect Service configuration - Update or add missing labels to pods - Remove or adjust NetworkPolicy rules - Confirm endpoints are correctly populated - Test connectivity again from another pod

Scenario 61 Kubernetes DNS resolution fails.

Problem / Symptoms	What to Check	Useful Commands	Fix / Resolution
 - Pods cannot resolve service names - DNS lookup fails with "no such host" - Intermittent DNS failures - Application cannot reach other services by name - External DNS resolution also failing (for external services) - nslookup or dig does not return results	- Check CoreDNS pods are running - Verify CoreDNS logs for errors - Test DNS resolution from inside a pod - Check if the service name and namespace are correct - Verify network policies or firewall rules - Check cluster DNS configuration and upstream DNS (for external lookups)	 <pre>kubectl get pods -n kube-system -l k8s-app=kube-dns kubectl logs -n kube-system -l k8s-app=kube-dns kubectl run -it --rm dns-test \ --image=busybox -- nslookup <service-name> cat /etc/resolv.conf kubectl get svc -A kubectl describe configmap coredns -n kube-system</pre>	 - Restart CoreDNS if needed - Fix CoreDNS configuration issues - Ensure correct service name and namespace - Check and update network policies - Verify upstream DNS servers - Increase CoreDNS resources if under-provisioned - Confirm cluster DNS settings are correct

Scenario 62 Application is reachable internally but not externally.

Problem / Symptoms	What to Check	Useful Commands	Fix / Resolution
 - Application works inside the cluster - Cannot be accessed from outside the cluster - Ingress or LoadBalancer not working - External IP is pending or not assigned - Firewall, security group or cloud network rules blocking - DNS or domain not resolving to the correct IP	- Verify the Service type (ClusterIP, NodePort, LoadBalancer, Ingress) - Check if external IP is assigned - Verify Ingress controller is running - Confirm DNS records are correct - Check cloud provider firewall and security group rules - Test connectivity from an external network	 <pre>kubectl get svc kubectl describe svc <svc> kubectl get ingress kubectl describe ingress <ing> kubectl get pods -n ingress-nginx nslookup <domain> curl -I http://<external-ip> kubectl get endpoints <svc></pre>	 - Use LoadBalancer or Ingress for external access - Ensure external IP is provisioned - Configure Ingress rules correctly - Update DNS records - Adjust cloud firewall / security group rules - Verify ingress controller is running and healthy - Test again from an external network

Key Takeaways

- Services rely on correct selectors, ports and endpoints
- CoreDNS is critical for service discovery
- Ingress and LoadBalancers enable external access
- Network policies and firewalls can block traffic
- Always test from inside and outside the cluster

Remember

- Check logs and events
- Verify configuration matches your intent
- Test step by step from pod to service to external
- Security rules can silently block traffic
- Document and monitor your networking setup

**BUILD
PRACTICE
GROW**



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

**LEARN
SHARE
GROW**

Kubernetes Configuration and Secrets

Troubleshoot configuration and secrets issues. Keep your workloads secure and correctly configured. Think like a DevOps engineer.

Scenario 63 ConfigMap change does not appear in the application.

Problem / Symptoms	What to Check	Useful Commands	Fix / Resolution
 - ConfigMap is updated but application still uses old values - Changes are not reflected in the running pod - Application needs a restart to pick up the new config - Volume mount or env var does not update automatically - Common with long-running pods	- Verify the ConfigMap is updated - Check how the ConfigMap is used (env var or volume mount) - Confirm if the application needs a restart to load changes - Check if the pod was recreated after the change - Verify the correct namespace and ConfigMap name - Inspect the application logs for configuration load events	 <pre>kubectl get configmap kubectl describe configmap kubectl get pods kubectl describe pod <pod> kubectl rollout restart deployment <name> kubectl logs <pod> kubectl exec -it <pod> -- printenv grep <VAR_NAME></pre>	 - Ensure the ConfigMap is updated correctly - Restart the deployment or pod to pick up changes - Use a rollout or rolling update - Verify the application watches for config changes (if supported) - Confirm the correct ConfigMap is mounted or referenced - Test and verify the new configuration is loaded

Scenario 64 Application cannot access its Secret.

Problem / Symptoms	What to Check	Useful Commands	Fix / Resolution
 - Application fails to start or cannot authenticate - Secret not found or access denied - Permission or RBAC issues - Wrong Secret name or namespace - Application error messages related to missing credentials	- Verify the Secret exists in the correct namespace - Check the Secret name and key - Confirm the pod has permission to access the Secret - Verify the Secret is mounted or referenced correctly - Check if the pod is using the correct service account - Inspect application logs for detailed errors	 <pre>kubectl get secret kubectl describe secret <name> kubectl get pod <pod> -o yaml kubectl describe pod <pod> kubectl get serviceaccount kubectl auth can-i get secret kubectl logs <pod></pre>	 - Create or update the Secret with correct data - Ensure correct namespace - Grant the necessary RBAC permissions - Mount the Secret correctly (as env var or volume) - Use the correct service account - Restart the pod and verify access

Scenario 65 Environment variable contains the wrong configuration.

Problem / Symptoms	What to Check	Useful Commands	Fix / Resolution
 - Application behaves unexpectedly - Environment variable has wrong value - Default or old configuration is used - Typo in variable name - Value in ConfigMap or Secret is incorrect	- Inspect the pod's environment variables - Verify the source (ConfigMap or Secret) - Check for typos in variable names - Confirm the correct namespace and key - Review the deployment manifest - Compare with expected values	 <pre>kubectl exec -it <pod> -- printenv sort kubectl describe pod <pod> kubectl get configmap <name> -o yaml kubectl get secret <name> -o yaml kubectl get deployment <name> -o yaml kubectl logs <pod></pre>	 - Correct the environment variable name or value - Update the ConfigMap or Secret with the right value - Redeploy or restart the pods - Verify the application reads the new value - Add validation or default values in the application - Test the application after the fix

Scenario 66 Secret is accidentally exposed in logs.

Problem / Symptoms	What to Check	Useful Commands	Fix / Resolution
 - Sensitive data appears in application logs - Secrets, tokens or credentials are visible - Logs are accessible to multiple users or external systems - Security risk and compliance violation	- Search logs for sensitive data - Check if the application logs environment variables - Review logging configuration - Verify if debug mode is enabled - Check CI/CD logs and monitoring tools - Identify where the secret is being printed	 <pre>kubectl logs <pod> grep -i "password" kubectl logs <pod> grep -i "token" kubectl logs <pod> grep -i "secret" kubectl describe pod <pod> kubectl get deployment <name> -o yaml kubectl get configmap <name> -o yaml</pre>	 - Remove sensitive data from logs - Disable debug logging in production - Use masked or redacted logging - Update application code to avoid logging secrets - Rotate any exposed credentials - Review and restrict log access permissions

Key Takeaways

- Use ConfigMaps for non-sensitive configuration
- Use Secrets for sensitive data
- Always verify namespace, names and permissions
- Restart or redeploy pods when required
- Monitor logs to avoid exposing sensitive information

Remember

- Configuration mistakes can cause application failures
- Secrets must always be protected
- Log files can leak sensitive data
- Follow the principle of least privilege
- Test changes in non-production first

**BUILD
PRACTICE
GROW**

Kubernetes Deployments and Health Checks

Troubleshoot deployment rollouts and health check issues. Keep your applications stable and highly available. Think like a DevOps engineer.

Scenario 67

Deployment rollout becomes stuck.

Problem / Symptoms	What to Check	Useful Commands	Fix / Resolution
 - Deployment does not complete the rollout - Pods stay in "Progressing" state - Some new pods are not ready - Old pods are not terminated - kubectl rollout status hangs or times out - Application remains unavailable or partially updated	- Check the deployment status and events - Verify the pod status and readiness - Check for image pull errors or crash loops - Verify resource availability on the nodes - Check readiness/liveness probes - Review the deployment strategy and replica settings	 <pre>kubectl get deployment <name> kubectl describe deployment kubectl get pods kubectl describe pod <pod> kubectl get events --sort-by='.lastTimestamp' kubectl rollout status deployment <name> kubectl rollout history deployment <name> kubectl logs <pod></pre>	 - Identify and fix the real cause (e.g. probes, image, resources) - Ensure new pods become ready - Check and correct resource requests/limits - Fix image pull or configuration issues - Adjust readiness probes if needed - Use kubectl rollout undo if necessary - Monitor the rollout until it completes successfully

Scenario 68

Readiness probe continuously fails.

Problem / Symptoms	What to Check	Useful Commands	Fix / Resolution
 - Pods start but are not ready - Readiness probe keeps failing - Pod is removed from the Service endpoints - Application is not reachable through the Service - Probe shows HTTP error, connection refused, or timeout - Logs may not show application errors	- Check pod status and events - Verify the readiness probe configuration - Test the probe endpoint inside the pod - Confirm the application is listening on the correct port - Check application logs for startup issues - Verify network policies or firewall rules are not blocking the probe	 <pre>kubectl describe pod <pod> kubectl get events --sort-by='.lastTimestamp' kubectl get pod <pod> -o yaml kubectl exec -it <pod> -- curl -v http://localhost:<port>/<path> kubectl logs <pod> kubectl get endpoints <service></pre>	 - Correct the probe path, port or protocol - Ensure the application is fully started before the probe runs - Increase initialDelaySeconds if needed - Fix application startup issues - Check network policies or firewall rules - Redeploy and verify the pod becomes ready

Scenario 69

Liveness probe causes healthy Pods to restart.

Problem / Symptoms	What to Check	Useful Commands	Fix / Resolution
 - Pods are repeatedly restarting - Liveness probe keeps failing - Pod appears healthy but is killed and restarted - Application logs show normal operation - Service experiences intermittent downtime - kubectl describe shows Liveness probe failed	- Check pod restart count and events - Review the liveness probe configuration - Verify if the probe endpoint is slow or resource intensive - Check application logs for probe-related issues - Confirm the probe is not too aggressive - Test the probe endpoint manually inside the pod	 <pre>kubectl get pods kubectl describe pod <pod> kubectl logs <pod> kubectl get events --sort-by='.lastTimestamp' kubectl exec -it <pod> -- curl -v http://localhost:<port>/<path> kubectl get pod <pod> -o yaml</pre>	 - Adjust the liveness probe settings (initialDelaySeconds, periodSeconds, timeoutSeconds, failureThreshold) - Use a more reliable endpoint - Ensure the probe does not trigger during normal high load - Fix application performance issues if any - Redeploy and monitor that pods remain stable - Consider using readiness probe for slow-starting applications



Key Takeaways

- Always check pod events and logs first
- Probes must match application behaviour
- Set realistic timings for health checks
- Monitor rollouts and pod restarts regularly
- Use rollbacks when needed
- Test from inside the cluster and from outside



Remember

- Small misconfigurations can cause big outages
- Health checks keep your applications reliable
- Monitor, alert and fix issues early
- Always validate after making changes
- Use rollout history and undo when necessary
- A stable cluster is a productive cluster

**BUILD
PRACTICE
GROW**



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

**LEARN
SHARE
GROW**

Kubernetes Capacity and Production Incidents

Troubleshoot cluster capacity, node issues and real production incidents. Keep your applications resilient and highly available. Think like a DevOps engineer.

Scenario 70

Cluster cannot schedule additional Pods.

Problem / Symptoms

- New Pods stay in Pending state
- Events show "Insufficient CPU" or "Insufficient memory"
- No suitable nodes available
- Resource requests may be too high
- Node taints, affinity or node selectors blocking scheduling
- Cluster has reached capacity

What to Check

- Check pod events and describe
- Verify node resources (CPU, memory, pods)
- Check for taints, node selectors and affinity rules
- Review ResourceRequests and limits
- Check if Cluster Autoscaler is working
- Look for quotas, limits or pod density issues

Useful Commands

```
>_
kubectl describe pod <pod>
kubectl get nodes
kubectl describe node <node>
kubectl get pods -A
kubectl describe pod <pod>
kubectl get events --sort-by='lastTimestamp'
kubectl top nodes
kubectl get resourcequota -A
```

Fix / Resolution

- Scale up cluster or node group
- Adjust resource requests and limits
- Remove or update taints, affinity or node selectors
- Enable or fix Cluster Autoscaler
- Optimize resource usage
- Ensure quotas and limits allow new Pods
- Re-apply and verify Pods are scheduled

Scenario 71

One node becomes NotReady.

Problem / Symptoms

- Node status is NotReady
- Pods on the node may be evicted or unavailable
- Kubelet may not be responding
- Network or system issues on the node
- Disk pressure, memory pressure or PID pressure
- Multiple nodes may be affected in severe cases

What to Check

- Check node status and conditions
- Describe the node for detailed status
- Check kubelet logs on the node
- Look for disk, memory or network issues
- Check recent events
- Verify cloud provider or infrastructure health

Useful Commands

```
>_
kubectl get nodes
kubectl describe node <node>
kubectl get events --sort-by='lastTimestamp'
journalctl -u kubelet -f
df -h && free -m
kubectl top node <node>
kubectl get pods -o wide
```

Fix / Resolution

- Fix the underlying node issue (disk, memory, network, etc.)
- Restart kubelet if needed
- If the node is unhealthy, cordon and drain it
- Replace or terminate the node (if required)
- Ensure cluster auto healing is working
- Verify new nodes join and become Ready

Scenario 72

Application latency increases after autoscaling.

Problem / Symptoms

- Application response time increases
- More replicas are running but performance is worse
- Increased CPU or memory usage
- Database or external dependencies are slow
- Network latency between pods
- Autoscaling may be causing resource contention

What to Check

- Check application metrics (latency, error rate, throughput)
- Verify HPA and scaling events
- Check resource usage of new pods
- Look for database or external service bottlenecks
- Review network latency and connection limits
- Compare performance before and after scaling

Useful Commands

```
>_
kubectl get hpa
kubectl describe hpa
kubectl get pods -o wide
kubectl top pods
kubectl get events --sort-by='lastTimestamp'
kubectl logs <pod>
kubectl get svc
kubectl get endpoints <svc>
```

Fix / Resolution

- Tune resource requests and limits
- Adjust HPA scaling policies and thresholds
- Optimize application and database performance
- Increase connection pool limits
- Use readiness probes to avoid sending traffic to unready pods
- Monitor and test performance during scaling

Scenario 73

A deployment causes widespread application errors.

Problem / Symptoms

- After a new deployment, multiple services show errors
- High error rate across the application
- Users cannot access key features
- Logs show exceptions or failures
- May be caused by a bad image, configuration or database change
- Impact across multiple environments or services

What to Check

- Identify when the issue started
- Check deployment and rollout history
- Review application logs and errors
- Compare with previous working version
- Check dependencies (database, APIs, config)
- Assess the blast radius (which services are affected)

Useful Commands

```
>_
kubectl rollout history deployment <name>
kubectl describe deployment <name>
kubectl get pods -l app=<label>
kubectl logs <pod> --tail=100
kubectl get events --sort-by='lastTimestamp'
kubectl get svc
kubectl get ingress
```

Fix / Resolution

- Rollback to the previous working version
- Fix the bad image or configuration
- Validate and test before re-deploying
- Use progressive rollout strategies (e.g. canary, blue/green)
- Monitor logs and metrics after fix
- Add automated tests and checks to prevent recurrence


Key Takeaways

- Monitor cluster capacity and node health regularly
- Use proper resource requests, limits and autoscaling
- Investigate logs, events and metrics for faster diagnosis
- Implement safe deployment strategies
- Always test and validate changes before production


Remember

- Small issues can quickly become major outages
- Monitor, alert and set proper thresholds
- Use rollbacks and progressive deployments
- Keep your cluster resilient and self-healing
- Learn from every incident and improve

**BUILD
PRACTICE
GROW**
**LEARN
SHARE
GROW**


@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

Cloud Compute and Networking

Troubleshoot cloud compute and networking issues. Keep your infrastructure secure, connected and highly available.
Think like a DevOps engineer.

Scenario 74 Cloud VM is running but cannot be reached.

Problem / Symptoms	What to Check	Useful Commands	Fix / Resolution
 - VM is in running state but cannot be accessed (SSH/RDP) - Connection times out or is refused - Instance is healthy but unreachable - Security group / firewall may be blocking traffic - Incorrect public IP, DNS or routing - Common in AWS, Azure and GCP	- Verify instance status and public/private IP - Check security group / firewall rules - Confirm correct port (22 for SSH, 3389 for RDP, or custom) - Check network ACL and route tables - Verify if the instance has a public IP or requires a bastion/VPN - Test connectivity using ping, telnet or nc from allowed source	 <pre>aws ec2 describe-instances aws ec2 describe-security-groups az vm show gcloud compute instances describe ping <instance-ip> nc -zv <instance-ip> <port> telnet <instance-ip> <port> ssh -v ubuntu@<instance-ip></pre>	 - Update security group / firewall to allow required port - Verify correct IP and DNS name - Check network ACL and route table configuration - Ensure instance has a public IP or use a bastion host/VPN - Confirm no OS-level firewall blocking traffic - Test again from an allowed network

Scenario 75 Private workload unexpectedly has internet exposure.

Problem / Symptoms	What to Check	Useful Commands	Fix / Resolution
 - A supposed private VM, container or service is accessible from the internet - Sensitive data or internal service is exposed - Public IP or open firewall rule found - Misconfigured load balancer or NAT gateway - Accidental exposure through misconfiguration - Security and compliance risk	- Check if the resource has a public IP - Review security group / firewall rules - Inspect load balancer settings - Check route tables and NAT gateway configurations - Look for open ports using network scanning tools - Review infrastructure as code (Terraform, CloudFormation, etc.) for misconfigurations	 <pre>aws ec2 describe-instances aws ec2 describe-security-groups aws elbv2 describe-load-balancers az network public-ip list gcloud compute addresses list nmap -Pn <public-ip> curl http://<public-ip>:<port> terraform plan</pre>	 - Remove public IP if not required - Restrict security group / firewall rules - Reconfigure or remove public load balancer - Update route tables and NAT settings - Apply least privilege access - Scan again to confirm the resource is no longer publicly accessible

Scenario 76 Application server cannot connect to its database.

Problem / Symptoms	What to Check	Useful Commands	Fix / Resolution
 - Application server is running but cannot connect to the database - Connection times out or is refused - Database is healthy but unreachable from the app server - Firewall, security group or network ACL blocking traffic - Incorrect database endpoint, port or credentials - DNS or routing issues between the servers	- Verify database is running and accessible - Check network connectivity between app server and database - Confirm security group / firewall rules allow database port - Validate database endpoint, port and credentials - Check DNS resolution and routing (VPC peering, VPN, etc.) - Review database logs for connection attempts	 <pre>nc -zv <db-endpoint> <port> telnet <db-endpoint> <port> ping <db-endpoint> nslookup <db-endpoint> dig <db-endpoint> aws rds describe-db-instances az mysql server show gcloud sql instances describe mysql -h <endpoint> -u <user> -p</pre>	 - Allow database port in security groups and firewall rules - Verify correct endpoint, port and credentials - Fix DNS or routing configuration - Ensure database is in the same VPC or connected via VPN/peering - Check database is accepting connections - Test the connection again from the application server - Monitor logs to confirm successful connections


Key Takeaways

- Always verify the resource is running and healthy
- Check network configuration (security groups, firewalls, routes)
- Use proper endpoints, ports and credentials
- Apply least privilege access
- Monitor logs and test connectivity from the correct network
- Regularly audit your cloud security settings


Remember

- A running instance does not mean it is reachable
- Private resources should never be exposed to the internet
- Network misconfigurations are a common cause of outages
- Always test from the same network as your application
- Security and monitoring prevent costly incidents
- Follow cloud provider best practices

**BUILD
PRACTICE
GROW**



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

**LEARN
SHARE
GROW**

IAM, Secrets, and Cloud Security

Troubleshoot identity, access, secrets and security issues in the cloud. Keep your environment secure and least privileged. Think like a DevOps engineer.

Scenario 77 Application receives AccessDenied.

Problem / Symptoms

- Application fails with AccessDenied error
- API call is not authorized
- Works in one environment but fails in another
- Missing or incorrect IAM permissions
- Trust policy or role assumption issues
- Common in AWS, Azure and GCP

What to Check

- Check the exact error message and denied action
- Verify the IAM policy attached to the user, role or service account
- Check resource-level permissions (ARN)
- Confirm the correct region and account
- Review trust policy if using assume-role
- Check for explicit deny in any policy (SCP, permission boundary, etc.)

Useful Commands

```
> aws sts get-caller-identity
> aws iam simulate-principal-policy
> aws iam get-role
> aws iam list-attached-role-policies
> aws sts assume-role
> aws iam get-policy-version
> az role assignment list
> gcloud projects get-iam-policy
> curl -v <cloud-api-endpoint>
```

Fix / Resolution

- Attach the correct permissions
- Update IAM policy to include the required actions and resources
- Ensure the correct role is used
- Remove explicit denies
- Verify trust relationships
- Test the action again
- Follow least privilege principle

Scenario 78 Engineer accidentally receives administrator privileges.

Problem / Symptoms

- Engineer has more permissions than required
- Full administrator access is granted
- Violation of least privilege principle
- Increases security risk
- May be caused by overly broad policies or group membership
- Common during onboarding or emergency access

What to Check

- Review the user's effective permissions
- Check group memberships and attached policies
- Look for managed policies like AdministratorAccess
- Review recent IAM changes or access requests
- Check if temporary access was not removed
- Audit who granted the access and why

Useful Commands

```
> aws iam get-user
> aws iam list-groups-for-user
> aws iam list-attached-user-policies
> aws iam list-attached-group-policies
> aws iam get-policy
> aws iam generate-service-last-accessed-details
> az ad user list --query
> gcloud projects get-iam-policy
```

Fix / Resolution

- Immediately remove unnecessary permissions
- Apply least privilege access
- Use specific managed or custom policies
- Implement approval process for elevated access
- Review and audit all user permissions regularly
- Use temporary access (with expiration) when needed

Scenario 79 Cloud credentials are exposed.

Problem / Symptoms

- Credentials found in code, logs or configuration files
- Access keys, tokens or secrets are publicly exposed
- Detected by security tools or manual review
- May lead to unauthorized access or data breach
- Common in GitHub, CI/CD logs, container images or shared files

What to Check

- Search code repositories for exposed credentials
- Check logs, build pipelines and environment files
- Look for hardcoded credentials
- Use secret scanning tools (e.g. trufflehog, git-secrets)
- Check if the keys are still active
- Review who had access and for how long

Useful Commands

```
> grep -i "AKIA" -r .
> grep -i "secret" -r .
> trufflehog filesystem .
> git-secrets --scan
> aws iam list-access-keys
> aws iam update-access-key --status Inactive
> gcloud iam service-accounts keys list
> az keyvault secret list
```

Fix / Resolution

- Immediately revoke or rotate the exposed credentials
- Remove credentials from code, logs and images
- Use a secrets manager (AWS Secrets Manager, Azure Key Vault, GCP Secret Manager)
- Implement secret scanning in CI/CD
- Use environment variables or IAM roles instead of hardcoding
- Monitor for future exposure

Scenario 80 Workload loses access after an IAM policy change.

Problem / Symptoms

- Previously working workload suddenly fails
- AccessDenied errors in logs
- API calls, database or storage access is blocked
- Recent IAM policy or permissions change
- May affect CI/CD, applications or automation

What to Check

- Review recent IAM policy changes
- Compare current policy with the previous version
- Check for missing actions or resources
- Verify if the workload uses a role, service account or user
- Check for explicit deny or SCP restrictions
- Test the required permissions using policy simulation

Useful Commands

```
> aws iam get-policy-version
> aws iam simulate-principal-policy
> aws cloudtrail lookup-events
> aws iam list-attached-role-policies
> kubectl describe serviceaccount
> az role assignment list
> gcloud projects get-iam-policy
> aws sts get-caller-identity
```

Fix / Resolution

- Restore the required permissions
- Update the IAM policy with correct actions and resources
- Remove unnecessary explicit denies
- Test the workload again
- Implement change management and testing process
- Use monitoring and alerting for IAM-related failures

Key Takeaways

- Use least privilege access for all identities
- Store secrets securely and never hardcode credentials
- Regularly audit IAM permissions and access keys
- Monitor for unauthorized access and policy changes
- Use automated tools to detect exposed credentials
- Implement proper change management for IAM updates

Remember

- AccessDenied usually means missing or incorrect permissions
- Administrator access increases the risk of compromise
- Exposed credentials can lead to a serious security incident
- Always use secrets managers and rotate keys regularly
- Test IAM policy changes before applying to production
- Security is everyone's responsibility

**BUILD
PRACTICE
GROW**

**LEARN
SHARE
GROW**



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

Terraform and Infrastructure as Code

Troubleshoot Terraform and Infrastructure as Code issues. Keep your infrastructure safe, consistent and reliable. Think like a DevOps engineer.

Scenario 81 terraform plan wants to destroy an important resource.

Problem / Symptoms	What to Check	Useful Commands	Fix / Resolution
 - terraform plan shows destructive changes - Important resources marked for deletion - Unexpected replacement (even though no intentional change was made) - Changes in configuration, provider version or state may be the cause - Risk of downtime or data loss if applied	- Review the plan output carefully - Check recent code changes - Look for provider version upgrades - Check for renamed or moved resources - Verify lifecycle settings (e.g. prevent_destroy) - Compare current state with real infrastructure	 <pre>terraform plan terraform plan -out=tfplan terraform show tfplan terraform state show <resource> terraform state list git diff (check recent changes) terraform providers terraform version</pre>	 - Review and understand why the resource is planned for destruction - If accidental, fix the configuration or set lifecycle { prevent_destroy = true } - Use terraform plan -out and review before applying - Restore from state if needed - Consider using terraform moved block for renamed resources - Apply only when you are sure changes are correct

Scenario 82 Terraform state becomes inconsistent with real infrastructure.

Problem / Symptoms	What to Check	Useful Commands	Fix / Resolution
 - terraform plan shows unexpected changes - Resources exist in the cloud but not in state (or vice versa) - Manual changes were made outside Terraform - Resources may be duplicated or missing - Drift between state file and real infrastructure - Can lead to accidental modifications or deletions	- Verify the current state file and resources - Compare state with real infrastructure - Check for manual changes made outside Terraform - Look for resources imported manually or via console - Identify missing or orphaned resources in state - Verify the correct workspace and backend	 <pre>terraform state list terraform state show <resource> terraform refresh terraform plan terraform import <resource> <id> terraform state rm <resource> terraform workspace show terraform workspace list</pre>	 - Run terraform refresh to update state - Import existing resources with terraform import - Remove stale resources from state with terraform state rm - Avoid making manual changes in the cloud - Use remote backend and state locking - Regularly review and audit infrastructure drift

Scenario 83 Two engineers attempt infrastructure changes simultaneously.

Problem / Symptoms	What to Check	Useful Commands	Fix / Resolution
 - Conflicting Terraform runs - State file is locked or becomes corrupted - One engineer's changes overwrite another's - Unexpected plan results - Resources end up in an inconsistent state - Can cause deployment failures or outages	- Check if state locking is enabled - Identify who is running Terraform - Review recent changes in the version control system - Look for overlapping changes in the same resources - Verify backend configuration (e.g. S3 with DynamoDB, Azure Storage, GCS) - Check for failed or incomplete runs	 <pre>terraform init terraform plan terraform apply terraform force-unlock <lock-id> terraform state list terraform state pull terraform workspace list git log (review recent changes)</pre>	 - Use a remote backend with state locking (e.g. S3 + DynamoDB) - Follow a change management process - Communicate with the team before making changes - Use feature branches and pull requests for Terraform code - If locked, verify no active runs and use terraform force-unlock - Re-run plan and apply after resolving conflicts


Key Takeaways

- Always review terraform plan before applying
- Use remote state with locking
- Avoid manual changes in the cloud
- Keep Terraform code in version control
- Use lifecycle rules to protect critical resources
- Follow team processes for infrastructure changes


Remember

- Infrastructure as code is powerful but can be dangerous
- Always review changes and understand the impact
- Use state locking to prevent concurrent runs
- Keep state files secure
- Communicate with your team
- Test changes in a non-production environment first


**BUILD
PRACTICE
GROW**

**LEARN
SHARE
GROW**

Infrastructure Changes and Recovery

Troubleshoot infrastructure changes, failed deployments and recovery scenarios. Keep your environments stable and resilient. Think like a DevOps engineer.

Scenario 84

Terraform deployment fails halfway through.

Problem / Symptoms

- ⚠ Terraform apply stops in the middle
- Some resources are created, others fail
- Environment is in a partial state
- Subsequent applies may fail
- Dependencies between resources may be broken
- Services may be unavailable or misconfigured

What to Check

- 1 Review the error message and failed resource
- 2 Check provider limits, quotas or permissions
- 3 Verify resource dependencies and order
- 4 Inspect the current state file
- 5 Check cloud console for partially created resources
- 6 Fix the issue and resume or re-apply safely

Useful Commands

```
> terraform apply
> terraform show
> terraform state list
> terraform state show <resource>
> terraform plan
> terraform apply -target=<resource>
> terraform refresh
> terraform taint <resource>
```

Fix / Resolution

- ✅ Identify and fix the root cause (e.g. quota, permissions, config)
- Re-run terraform apply after fixing
- Use -target to complete specific resources if needed
- Clean up failed or partial resources manually if required
- Verify the final state and infrastructure is healthy
- Implement better error handling and testing for future changes

Scenario 85

Manual cloud change creates configuration drift.

Problem / Symptoms

- ⚠ Resources are changed manually in the cloud console
- Terraform plan shows unexpected changes
- Infrastructure no longer matches code
- Manual changes may cause inconsistent environments
- Future applies may overwrite manual changes
- Hard to track who made the change

What to Check

- 1 Compare Terraform state with real infrastructure
- 2 Identify what was changed manually
- 3 Check cloud console and audit logs
- 4 Review recent activity and user actions
- 5 Look for drift in critical resources (e.g. security groups, IAM, network settings)
- 6 Decide whether to import, revert or update code

Useful Commands

```
> terraform plan
> terraform show
> terraform state list
> terraform import <resource> <id>
> terraform refresh
> aws cloudtrail lookup-events
> aws resourcegroupstaggingapi get-resources
> az resource list | grep <name>
> gcloud compute instances list
```

Fix / Resolution

- ✅ Revert manual changes or update Terraform code
- Import manually created resources if needed
- Use terraform refresh to update state
- Implement change control processes
- Use IAM policies to restrict manual changes
- Regularly run drift detection as part of operations

Scenario 86

Infrastructure module update breaks production.

Problem / Symptoms

- ⚠ After updating a module, resources fail or behave unexpectedly
- Services become unavailable or misconfigured
- Breaking changes in the new module version
- Outputs or variables may have changed
- Dependencies between modules may conflict
- Production environment is affected

What to Check

- 1 Review the module changelog and version differences
- 2 Check terraform plan output for unexpected changes
- 3 Identify deprecated or removed variables/outputs
- 4 Verify provider and module version compatibility
- 5 Check logs and resource status after apply
- 6 Test the update in a staging environment

Useful Commands

```
> terraform plan
> terraform init -upgrade
> terraform providers
> terraform state list
> terraform show
> terraform plan -out=tfplan
> terraform apply tfplan
> git diff (module changes)
```

Fix / Resolution

- ✅ Rollback to the previous working module version
- Update code to match new module requirements
- Test module updates in a non-production environment
- Pin module versions to avoid unexpected changes
- Review module documentation and upgrade guides
- Implement automated testing for infrastructure changes

Scenario 87

Critical resource is accidentally destroyed.

Problem / Symptoms

- ⚠ Important resource (e.g. DB, load balancer, storage) is deleted
- Service is down or degraded
- Data may be lost or inaccessible
- Deletion may be due to a misconfig, wrong workspace or human error
- Recovery may be time sensitive
- Business impact can be significant

What to Check

- 1 Confirm what was deleted and when
- 2 Check audit logs and activity history
- 3 Look for backups, snapshots or recovery options
- 4 Verify if the resource can be restored (e.g. from snapshots)
- 5 Review the cause (plan output, wrong workspace, etc.)
- 6 Assess the impact and recover or recreate the resource

Useful Commands

```
> aws cloudtrail lookup-events
> aws rds describe-db-instances
> aws rds restore-db-instance-from-snapshot
> aws s3 ls (check backups)
> az resource list --show-deleted
> gcloud compute instances list --filter="status:TERMINATED"
> terraform state list
```

Fix / Resolution

- ✅ Restore from backups or snapshots if available
- Recreate the resource using Terraform or IaC
- Investigate and fix the root cause
- Implement stricter review and approval processes
- Use Terraform lifecycle rules (e.g. prevent_destruction)
- Enable monitoring and alerts for critical resources


Key Takeaways

- Always review plan output carefully
- Use version control and change approvals
- Detect and prevent configuration drift
- Test module updates in non-production
- Keep backups and recovery plans
- Implement safeguards for critical resources


Remember

- Infrastructure changes can have a big impact
- Always validate before applying
- Use monitoring and audit logs
- Have a rollback or recovery plan
- Follow the principle of least privilege
- Test in a staging environment first

**BUILD
PRACTICE
GROW**
**LEARN
SHARE
GROW**

Monitoring and Alerting

Troubleshoot monitoring and alerting issues. Keep your systems observable, alerting trustworthy and actionable.
Think like a DevOps engineer.

Scenario 88

CPU alert repeatedly fires but users see no problem.

Problem / Symptoms	What to Check	Useful Commands	Fix / Resolution
 - CPU alert keeps firing but application seems healthy - Users report no issues - High CPU usage in monitoring tools - May be caused by background jobs, cron jobs or non-critical processes - Alert noise and alert fatigue - Common in Kubernetes, VMs or cloud services	- Check which processes are using CPU - Verify if the high CPU is sustained or just a spike - Check if the affected service is critical - Compare with application metrics (latency, error rate, throughput) - Review recent deployments or scheduled jobs - Check if the alert threshold is too sensitive	 <pre>top htop ps aux --sort=-%cpu kubectl top pods kubectl top nodes kubectl describe pod <pod> aws cloudwatch get-metric-statistics curl -s http://<service>/health promql:rate(container_cpu_usage_seconds_total[5m])</pre>	 - Identify and confirm the increased CPU source - Adjust alert thresholds and evaluation period - Exclude non-critical processes (e.g. batch jobs) - Use multiple signals (CPU + latency + errors) - Tune alerts to reduce noise - Document the expected CPU patterns - Implement alert suppression during known maintenance windows

Scenario 89

Application fails without triggering an alert.

Problem / Symptoms	What to Check	Useful Commands	Fix / Resolution
 - Application is down or behaving incorrectly - No alert was triggered - Users report issues but monitoring is silent - Critical metric or log not being monitored - Alert rules may be missing or misconfigured - May be caused by new service, deployment change or blind spots in monitoring	- Verify what is actually monitored for the application - Check if the right metrics, logs and health checks exist - Review alert rules and thresholds - Check if the service was recently deployed or changed - Validate that alerts are routed correctly - Look for gaps in monitoring coverage (new endpoints, new environments)	 <pre>kubectl get pods kubectl get svc kubectl logs <pod> curl -s http://<service>/health promql: up promql: rate(http_requests_total[5m]) aws cloudwatch describe-alarms kubectl get endpoints kubectl get ingress tail -f /var/log/app.log</pre>	 - Create or update monitoring for the missing component - Add health checks and synthetic checks - Set up alerts for critical metrics (latency, errors, availability) - Ensure alert routing and notifications are working - Include new services and environments in monitoring - Test alerts to confirm they fire - Implement a monitoring onboarding checklist for new services

Scenario 90

Monitoring system generates hundreds of alerts during one incident.

Problem / Symptoms	What to Check	Useful Commands	Fix / Resolution
 - Hundreds of alerts fire at once - Alert storm and notification flooding - Difficult to identify the real root cause - Multiple services appear affected - Caused by one underlying issue (e.g. network, database, cloud service) - Leads to alert fatigue and slower incident response	- Identify the first alert and timeline of events - Look for a common dependency (e.g. database, network, cloud service) - Check the scope and blast radius - Correlate alerts across services - Verify if alert rules have dependencies or lack suppression - Check incident communication and escalation flow	 <pre>kubectl get events --sort-by=.lastTimestamp kubectl get pods -A kubectl get nodes aws cloudwatch describe-alarms promql: changes(up[5m]) promql: sum by (alertname) (ALERTS) kubectl describe node <node> kubectl get svc -A kubectl logs <pod> --previous</pre>	 - Identify and fix the root cause - Implement alert grouping and deduplication - Use alert correlation tools (e.g. Alertmanager, PagerDuty) - Add dependency mapping between services - Set up proper alert suppression during incidents - Improve alert thresholds and reduce noise - Run a post-incident review and tune alerting rules

 **Key Takeaways**

- Monitor the right things, not everything
- Use meaningful and actionable alerts
- Reduce alert noise and alert fatigue
- Test your monitoring and alerts regularly
- Correlate alerts to find the real root cause
- Keep improving observability and incident response

 **Remember**

- An alert is only useful if it is actionable
- Alert storms hide the real problem
- Monitor availability, performance and errors
- Use alert grouping and suppression
- Always test new alerts
- Learn from every incident and tune your monitoring

**BUILD
PRACTICE
GROW**

**LEARN
SHARE
GROW**



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

Logs, Metrics, and Traces

Troubleshoot observability issues using logs, metrics and traces. Find the real problem, not just symptoms.
Think like a DevOps engineer.

Scenario 91

Metrics show high latency, but logs show no errors.

Problem / Symptoms



- Metrics show high latency or slow response time
- Logs show no errors
- Users experience slowness but no failures
- May be caused by external dependency, database, network or resource saturation
- Common in microservices and cloud environments

What to Check

- 1 Verify if latency is real and consistent across instances
- 2 Check application and dependency metrics (database, external APIs)
- 3 Review network metrics and resource usage (CPU, memory, I/O)
- 4 Check for increased traffic or unusual load
- 5 Analyze traces to find slow spans and bottlenecks
- 6 Compare latency by region, instance or path

Useful Commands



```
kubectl top pods
kubectl top nodes
kubectl describe hpa
curl -w "time_connect: %(time_connect)\n" -o /dev/null http://localhost:8080/
time_total: %(time_total)\n" <url>
aws cloudwatch get-metric-statistics
kubectl logs <pod>
kubectl get svc, ep
jaeger query
promql: histogram_quantile()
```

Fix / Resolution



- Identify the real bottleneck (using metrics and traces)
- Check external services and database performance
- Scale resources if needed
- Optimize slow queries or API calls
- Fix network issues
- Add tracing if not available
- Set proper latency alerts
- Validate after the fix to ensure latency returns to normal

Scenario 92

Logs indicate failures but cannot identify the affected request.

Problem / Symptoms



- Logs show errors or failures
- Cannot identify which user request or transaction failed
- Logs do not have request IDs or correlation IDs
- Difficult to trace the issue across services
- Common in distributed systems and microservices

What to Check

- 1 Check if logs include request ID, trace ID or correlation ID
- 2 Verify log format and fields (e.g. timestamp, service, request_id)
- 3 Look at logs from all related services
- 4 Check if tracing is enabled (Distributed tracing)
- 5 Correlate logs with metrics and traces
- 6 Review recent changes in logging configuration

Useful Commands



```
kubectl logs <pod> | grep ERROR
kubectl logs <pod> | grep <request-id>
kubectl logs <pod> --since=1h
grep -r "request_id" /var/log/
jaeger query
aws logs insights
kubectl get pods -l app=<name>
kubectl describe pod <pod>
```

Fix / Resolution



- Enable request/correlation IDs in logs
- Standardize log format with useful fields (JSON)
- Propagate trace IDs across all services
- Use distributed tracing (e.g. Jaeger)
- Centralize logs in a log aggregator (e.g. ELK, Loki, CloudWatch)
- Update logging libraries and configuration
- Reproduce and verify you can trace the full request

Scenario 93

Distributed request becomes slow across several services.

Problem / Symptoms



- End-to-end request is slow
- Multiple services are involved
- No single service shows a clear error
- Latency increases across the whole request path
- Difficult to identify which service is the root cause
- Common in microservices and distributed systems

What to Check

- 1 Use tracing to visualize the full request flow
- 2 Identify which service or span has the highest latency
- 3 Check logs and metrics for each service in the trace
- 4 Verify if there are retries, timeouts or queue delays
- 5 Check database and external API calls
- 6 Compare with normal/healthy requests

Useful Commands



```
jaeger query
kubectl logs <pod>
kubectl top pods
promql: rate(http_request_duration_seconds_sum(5m))
kubectl describe svc <service>
curl -w "time_total: %(time_total)\n" -o /dev/null http://localhost:8080/
aws xray get-trace-summaries
kubectl get pods -o wide
```

Fix / Resolution



- Find and fix the slowest service in the trace
- Optimize database or external API calls
- Reduce unnecessary retries and timeouts
- Improve caching and connection pooling
- Scale the affected service
- Set up distributed tracing if not already enabled
- Test again and confirm latency is back to normal

Scenario 94

Monitoring data disappears during an outage.

Problem / Symptoms



- Metrics, logs or traces suddenly stop appearing
- Monitoring dashboard is empty or shows gaps
- Alerts may also stop firing
- Causes blind spots during an incident
- Common due to agent failure, network issues or infrastructure outage

What to Check

- 1 Check if monitoring agents are running
- 2 Verify agent logs and resource usage (CPU, memory, disk)
- 3 Check network connectivity to the monitoring backend
- 4 Confirm if the backend service is healthy
- 5 Look for recent changes or deployments
- 6 Verify if data is delayed or lost

Useful Commands



```
kubectl get pods -n monitoring
kubectl logs <agent-pod>
kubectl top pods -n monitoring
curl -v <monitoring-endpoint>
systemctl status <agent>
aws cloudwatch describe-alarms
promql: up
kubectl get events --sort-by=.lastTimestamp
```

Fix / Resolution



- Restart failed monitoring agents
- Fix network connectivity issues
- Increase resource limits if needed
- Restore or failover the monitoring backend
- Implement high availability for monitoring systems
- Set up meta-monitoring to alert when monitoring stops
- Backfill or accept data gaps and document the incident

Key Takeaways

- Use logs, metrics and traces together
- Add request IDs and enable distributed tracing
- Monitor your monitoring systems
- Set meaningful alerts and reduce noise
- Always test observability during incidents
- Good visibility reduces MTTR

Remember

- Metrics show the symptom, logs give details, traces show the full story
- No logs or metrics often means a bigger problem
- Observability is part of production readiness
- Invest in good instrumentation and alerting
- A blind monitoring system can make an incident worse

**BUILD
PRACTICE
GROW**

Database and Dependency Failures

Troubleshoot database and external dependency issues. Find the real cause, not just the symptom.
Think like a DevOps engineer.

Scenario 95

Application exhausts database connections.

Problem / Symptoms

- Application throws "too many connections" or "connection pool exhausted"
- New requests fail or time out
- Database becomes slow or unresponsive
- Other services may also be affected
- Connections remain in "sleep" state
- Common during traffic spikes or connection leaks

What to Check

- Check current connection count and limits
- Look for long-lived or idle connections
- Review application connection pool settings
- Check for connection leaks in the code
- Identify sudden traffic spikes or scaling events
- Verify database max connections configuration

Useful Commands

```
>_
# Check active connections
SELECT count(*) FROM pg_stat_activity;
# Check connection details
SELECT state, count(*)
FROM pg_stat_activity
GROUP BY state;
# Check max connections
SHOW max_connections;
# For MySQL
SHOW PROCESSLIST;
```

Fix / Resolution

- Increase database connection limit (if safe)
- Fix connection leaks in the application
- Optimize connection pool settings (e.g. HikariCP, PgBouncer)
- Close idle connections
- Implement proper retry and backoff logic
- Add monitoring and alerts for connection usage
- Consider using a connection pooler (PgBouncer, RDS Proxy, etc.)

Scenario 96

External API becomes slow or unavailable.

Problem / Symptoms

- Application experiences timeouts or slow responses
- Users see errors or degraded functionality
- External API returns 5xx errors or no response
- May affect multiple services depending on the API
- Can be caused by the third party or network issues
- Often leads to cascading failures in your system

What to Check

- Check API status page (from the provider)
- Test the API directly using curl or a tool like Postman
- Check latency and error rates over time
- Verify network connectivity (DNS, routing, firewall)
- Look for recent changes in your application (e.g. new version, higher traffic)
- Check if you have proper timeouts and retries configured

Useful Commands

```
>_
# Test the API
curl -v https://api.example.com
# Check DNS resolution
nslookup api.example.com
# Test network connectivity
telnet api.example.com 443
# Check response time
curl -w "Time: %(time_total)s\n"
-o /dev/null -s https://api.example.com
```

Fix / Resolution

- Confirm if the issue is on the provider side
- Implement retries with exponential backoff
- Set appropriate timeouts
- Use a circuit breaker (e.g. in your application)
- Add fallback or degraded mode (e.g. cached data)
- Monitor external dependencies with alerts
- Communicate with the provider if the issue persists

Scenario 97

Database latency increases immediately after deployment.

Problem / Symptoms

- Database response time increases after a new deployment
- Application becomes slow
- Queries take longer to complete
- May cause timeouts or errors in the application
- Often related to new queries, changes in traffic patterns, or missing indexes
- Users experience degraded performance

What to Check

- Compare database metrics before and after deployment
- Look for new or changed queries in the application
- Check for missing indexes or inefficient queries
- Review resource usage (CPU, memory, disk I/O)
- Check for locks or blocked queries
- Verify if the deployment changed connection pool settings or increased load

Useful Commands

```
>_
# Check slow queries (PostgreSQL)
SELECT query, mean_time, calls
FROM pg_stat_statements
ORDER BY mean_time DESC
LIMIT 10;
# Check active queries
SELECT * FROM pg_stat_activity
WHERE state = 'active';
# Check locks
SELECT * FROM pg_locks;
```

Fix / Resolution

- Identify and optimize slow queries
- Add missing indexes
- Rollback or adjust the deployment if it introduced inefficient queries
- Check and tune database configuration (CPU, memory, IOPS)
- Review connection pool settings
- Monitor performance after fixes
- Consider using query analysis tools (e.g. pg_stat_statements, Performance Insights)

Key Takeaways

- Monitor database and external dependencies.
- Set proper timeouts, retries and circuit breakers.
- Compare metrics before and after deployments.
- Identify the real cause, not just the symptom.
- Have a rollback plan and communicate early.

Remember

- Database and external API issues can cascade. Monitoring and alerting are critical.
- Optimize queries and use indexes.
- Use connection pooling and proper limits.
- Always test changes in a non-production environment.

**BUILD
PRACTICE
GROW**

Major Production Incident

Real-world major incident scenarios, troubleshooting steps and recovery actions.
Think like a DevOps engineer.

Scenario 98 Traffic spike causes cascading service failures.

Problem / Symptoms	What to Check	Useful Commands	Fix / Resolution
 - Sudden spike in traffic (overload) - Multiple services start failing - High error rates (5xx, timeouts) - Increased latency across the platform - Dependent services fail one after another (cascading failure) - Infrastructure becomes unhealthy (high CPU, memory, connection usage) - Users experience outages or degraded performance - May be triggered by real user growth, marketing event, malicious traffic or bot traffic	- Check traffic patterns and metrics (RPS, latency, error rate) - Identify which service failed first - Check infrastructure metrics (CPU, memory, network, connections) - Review recent deployments or configuration changes - Check autoscaling (is it working, any limits reached?) - Inspect logs for errors, timeouts or throttling - Verify database and external dependencies - Look for cascading failures in dependent services	 <pre>kubectl top nodes kubectl top pods kubectl get hpa kubectl describe hpa kubectl get pods -A kubectl get events --sort-by=.lastTimestamp kubectl logs <pod> kubectl get svc curl -w "time_total: %(time_total)\n" aws cloudwatch get-metric-statistics kubectl describe pod <pod> kubectl get endpoints kubectl get ingress</pre>	 - Scale up capacity (manually or fix autoscaling) - Increase rate limits or enable throttling - Use load balancing and caching (e.g. CDN) - Fix the root cause of the spike (e.g. bad query, bot traffic) - Implement circuit breakers and bulkheads - Optimise database and external API usage - Add monitoring and alerts for traffic anomalies - Conduct a post-incident review and improve scalability and resilience

Scenario 99 A seemingly small deployment causes a company-wide outage.

Problem / Symptoms	What to Check	Useful Commands	Fix / Resolution
 - A small code or configuration change is deployed - Unexpected behaviour across multiple services - Large number of errors or service failures - Critical user journeys stop working - Incident affects the entire platform or company - May be caused by shared libraries, configuration changes, database changes or infrastructure updates - Users cannot access core services - Business impact is significant	- Identify what changed (recent deployments, configurations, feature flags) - Check deployment history and rollback options - Review logs across all affected services - Check shared dependencies (libraries, databases, APIs) - Verify configuration changes (environment variables, secrets, infrastructure) - Look for error patterns across multiple services - Check database and infrastructure changes - Determine the blast radius (how many services are affected)	 <pre>kubectl rollout history deployment kubectl rollout undo deployment kubectl get deployments -A kubectl get events --sort-by=.lastTimestamp kubectl logs <pod> kubectl describe deployment git log --oneline git diff <previous> <current> kubectl get configmaps kubectl get secrets kubectl get pods -A kubectl get svc -A kubectl get ingress -A</pre>	 - Quickly identify and rollback the change - Restore previous stable version or configuration - Check and fix dependent services - Validate database and infrastructure state - Ensure feature flags and configurations are correct - Communicate with stakeholders and users - Monitor system recovery closely - Conduct a detailed post-incident review to prevent recurrence - Implement better testing, canary releases and change management

 **Key Takeaways**

- Understand system dependencies.
- Monitor for traffic spikes and unusual patterns.
- Always review changes before and after deployment.
- Use rollback and feature flags to reduce risk.
- Conduct post-incident reviews and implement improvements.

 **Remember**

- Small changes can have a big impact.
- Monitor, detect and respond quickly.
- Communicate clearly during incidents.
- Learn from every incident.
- Build resilient, scalable and observable systems.

**BUILD
PRACTICE
GROW**

The Complete DevOps Interview Challenge

A real-world, end-to-end scenario to test your DevOps thinking, troubleshooting and communication skills.
Think like a DevOps engineer.

Scenario 100 Production application is intermittently unavailable after a new deployment.

Scenario Overview

- Users report that the production application is intermittently unavailable after a new deployment.
- Some Pods are restarting.
- Latency has increased.
- Database connections are nearly exhausted.
- The load balancer reports unhealthy targets.
- CI/CD reports that the deployment technically succeeded.

Your Task

Work through the incident using a structured approach:

- 1 SYMPTOM
- 2 IMPACT
- 3 EVIDENCE
- 4 HYPOTHESIS
- 5 DIAGNOSTICS
- 6 ROOT CAUSE
- 7 SAFE FIX
- 8 VERIFICATION
- 9 ROLLBACK
- 10 PREVENTION



Skills Being Tested

- ✓ Systematic troubleshooting
- ✓ Use of logs, metrics and traces
- ✓ Kubernetes, cloud and networking knowledge
- ✓ Database understanding
- ✓ CI/CD and deployment analysis
- ✓ Incident communication
- ✓ Decision making under pressure
- ✓ Risk management
- ✓ Rollback and recovery
- ✓ Long term improvement mindset

1 SYMPTOM

- What exactly is happening?
- Be specific and avoid assumptions.
- Examples: intermittent errors, Pod restarts, high latency, unhealthy targets, DB connection errors.

2 IMPACT

- Who is affected?
- What features or services are down or degraded?
- What is the business impact?
- Is it a full outage or partial degradation?

3 EVIDENCE

- Collect evidence from multiple sources:
- kubectl (Pods, events, describe)
- Application logs
- Metrics (CPU, memory, latency, DB connections)
- Load balancer / ingress / service status
- CI/CD deployment details
- Database logs and connection metrics

4 HYPOTHESIS

- Based on the evidence, what do you think is causing the issue?
- List possible causes (be open minded).
- Prioritize the most likely cause.

5 DIAGNOSTICS

- Run targeted checks to confirm or reject your hypothesis.
- Use appropriate tools and commands (kubectl, cloud CLI, database tools, monitoring, tracing).
- Look for patterns and correlate data across systems.

6 ROOT CAUSE

- What is the actual root cause?
- Explain clearly with evidence.
- Why did it happen after the deployment?
- Was it a code change, configuration change, resource limit, dependency issue, database issue, or something else?

7 SAFE FIX

- What is the safest and most effective way to fix the issue?
- Consider short term and long term fixes.
- Minimize risk and avoid further impact.
- Communicate the plan if this is a real incident.

8 VERIFICATION

- How do you confirm the issue is resolved?
- Check application functionality.
- Verify metrics, logs and health checks.
- Ensure Pods are stable.
- Confirm database connections are normal.
- Validate load balancer targets are healthy.

9 ROLLBACK

- If the fix does not work or makes things worse, how will you roll back?
- What is your rollback plan?
- How do you minimize downtime?
- Have you tested the rollback process?

10 PREVENTION

- What will you do to prevent this from happening again?
- Update monitoring and alerting (e.g. add better alerts).
- Improve testing in CI/CD (e.g. integration tests, canary deployments).
- Review deployment and configuration management.
- Add safeguards (e.g. resource limits, connection pooling, health checks).
- Document the incident and share learnings with the team.

Key Takeaways

- ✓ Think systematically
- ✓ Use data, not assumptions
- ✓ Consider the bigger picture
- ✓ Communicate clearly
- ✓ Be prepared to roll back
- ✓ Focus on long term improvements
- ✓ This is how real DevOps engineers work

**BUILD
PRACTICE
GROW**

**LEARN
SHARE
GROW**



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta